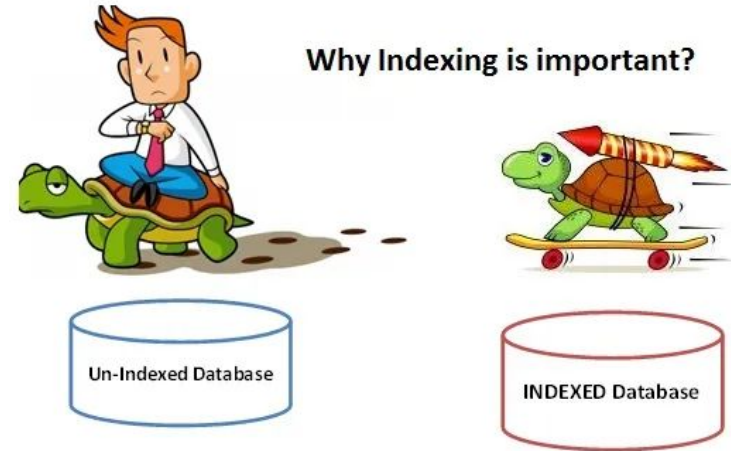
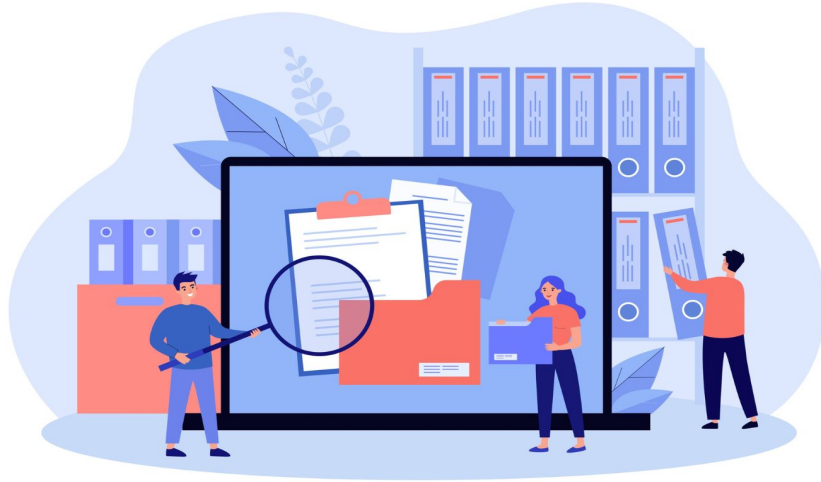


Information Retrieval (IR)

Unit 1 outcome: Students will **understand and implement the basic concepts in indexing and its compressed construction.**



References

1. “Modern Information Retrieval: The Concepts and Technology Behind Search”, Ricardo Baeza-Yates and Berthier Ribeiro-Neto, Second Edition, (Pearson Education India, 2010)
2. “Introduction to Information Retrieval”, C. Manning, P. Raghavan, and H. Schütze, (Cambridge University Press, 2008)
3. Internet Resources

Buzzwords...

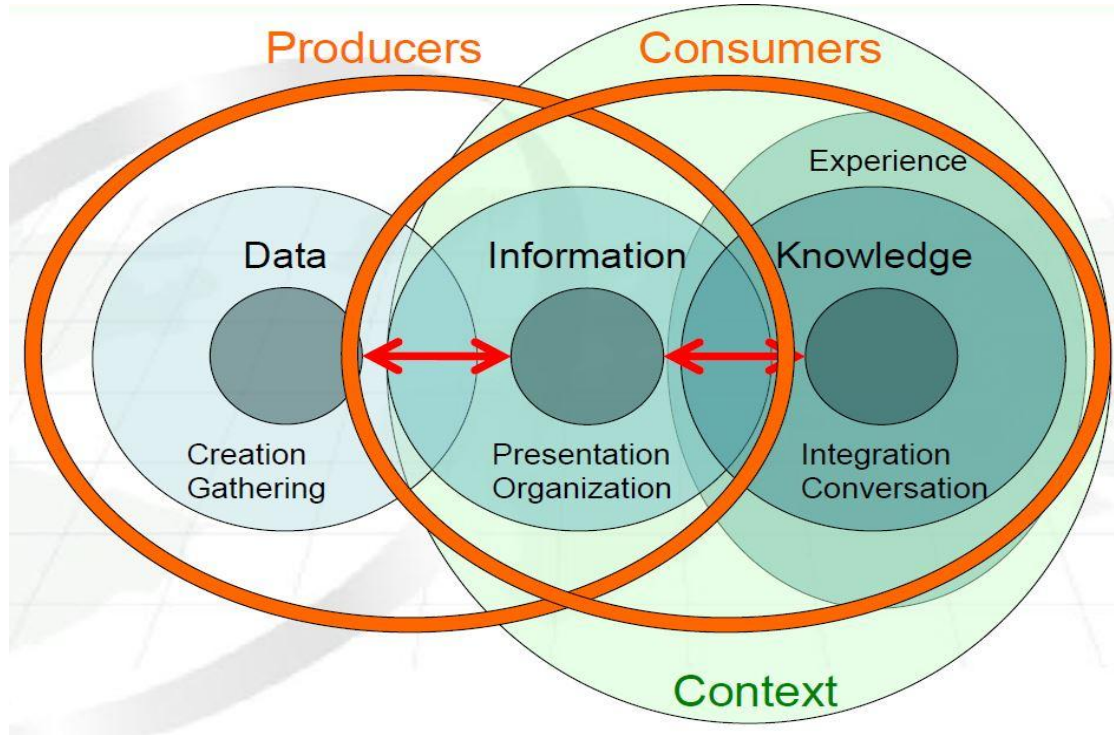
Data

Search

Information

Retrieval

Information Retrieval?



Data	Information
Raw and Unprocessed	Processed
Unorganized	Organized
Not context	Has some context

- IR Definitions:
 - IR deals with **representation, organization, storage** of and **access** to information items.
 - IR is **finding material** (usually documents) of an **unstructured** nature (usually text) that satisfies an information need from within **large collections** (usually stored on computers).
- IR is meant for Unstructured or Semistructured records:
 - Data that does not have clear, semantically overt, easy for computer structure.
- Point of views of study:
 - **Computer-centered**: building of efficient indexes, processing user queries with high-performance and developing ranking-algorithms.
 - **Human-centered**: study the behaviour of user, understanding their need and of determining how such understanding affects organization and operations of system.

- Early developments:
 - 3000-2500 BC: Oldest library “Fertile Crescent” currently northern Syria
 - 1950 : Hans Peter Luhan, Eugene Garfield, and Philip Bagley and Calvin Moores coined the term “Information Retrieval”
 - 1955: Allen Kent published paper describing precision and recall metrics
 - 1963: Joseph Becker and Robert Hayes published first book on “Information Retrieval”
 - 1978: First ACM conference on Information Retrieval in New York
 - 1983: Salton published “Introduction to Modern Information Retrieval” book
- Contemporary IR research includes:
 - Modeling, Systems architecture
 - Web search, Text classification
 - User interfaces, Data visualization
 - Filtering, languages

IR in Libraries and Digital Libraries

- First generation: card catalogs searching restricted to author names and titles.
- Second generation: increase search functionality was added to include subject headings, key words and query operators.
- Third generation: focus has been improved graphical interfaces, electronic forms, hypertext features and open system architectures.

IR at center of stage

- In 1990's:
 - Web is becoming a universal repository for human knowledge.
 - Finding useful information of interest is a tedious and difficult task.
 - Main obstacle is absence of well defined underlying data model for Web.
- IR has gained a place with other technologies at the center of stage.

IR problem

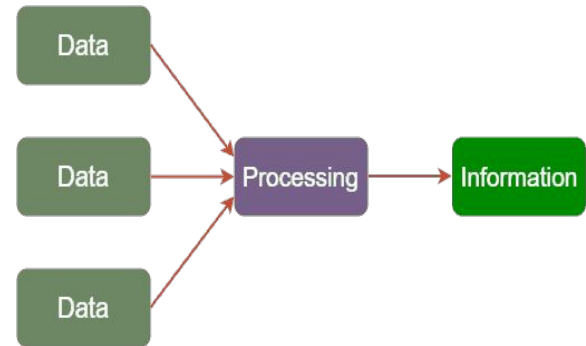
- Users of modern IR systems have information need of **varying complexity**.
- Examples:
 - Find all documents that address the role of Federal Government of financing operation of National Railroad Transportation Corporation.
 - Find all pages containing information on college tennis teams which: (1) are maintained by an university in the USA and (2) participate in NCAA tennis tournament.
 - Find all students in each class: (1) whose attendance percentage greater than 90% and having 9 CGPA (2) able to develop simple applications.



IR problem ...

- User must first translate this information into “**Query**” which can be processed by search engine (or IR System).
- Translation yields set of **key words or index terms** which summarizes the description of information need.
- Emphasis is on the retrieval of “**information**” instead of “**data**”.

Data	Information
Facts and statistics collected together for reference or analysis	Facts provided or learned about something or someone
Raw and Unprocessed	Processed
Unorganized and has no context	Organized and has some context

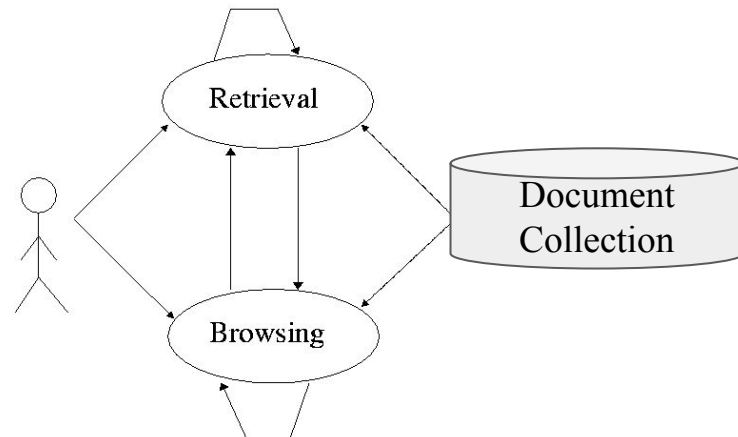


IR problem ...

- **Interpretation** of document contents of information items according to user query.
- **IR Problem:**
 - The primary goal of IR system is to **retrieve all documents** that are relevant to user query while retrieving **few non-relevant documents as possible**.
- **Relevance** is of central importance in IR.
 - It depends on **task being solved** and its **context**
 - It can change with **time or location or device**

User's task

- Translate their information need into a query in language provided by the system.
- User is **searching or querying** for information of their interests.
- Interest that is poorly defined or inherently broad:
 - Documents about car racing?
- Tasks might be of two distinct types:
 - Searching or querying
 - Browsing or navigating



Data retrieval vs Information retrieval

Data retrieval	Information retrieval
Aims at retrieving all objects which satisfy clearly defined conditions such as regular expression or relational algebra expression.	Aims at retrieving information about a subject than with retrieving data which satisfies a given query.
Deals with data that has a well defined structure and semantics .	Deals with natural language text which is not always well structured and semantically ambiguous .
Single erroneous object among thousand retrieved objects means total failure .	The retrieved objects might be inaccurate and small errors are likely to go unnoticed.

Software architecture of the IR system

- **Retrieval & Ranking process**

- It consists of retrieving documents that satisfy either user query (searching) or click in hyperlink (browsing).
- Query is expanded (system query) and then processed against index to retrieve subset of all documents.
- Retrieved documents are ranked and top documents are returned to user.
- Ranking is to identify documents that are most likely to be considered relevant by user.
- Evaluating quality of answer set is key step for improving IR system
- Common evaluation procedure consists of comparing set of results produced with results suggested by human specialists.
- Collect feedback from users and use this information to change results.

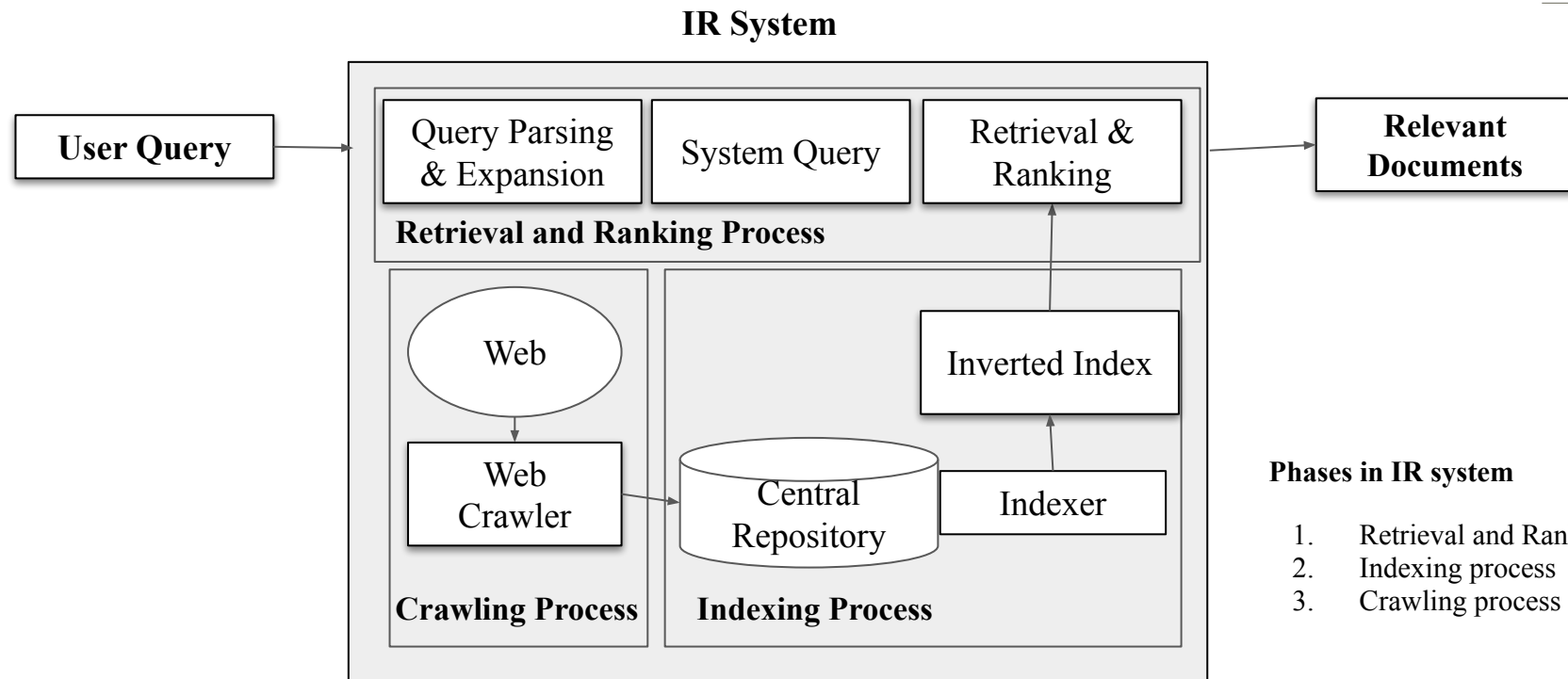
- **Indexing process**

- Central repository needs to be indexed for fast retrieval and ranking.
- Most used index structure is an “Inverted index”

- **Crawling process**

- Assemble document collection which can be private or crawled from the web
- Crawler is responsible for collecting the documents.
- Stored in disk storage usually referred as **Central repository**.

IR System ...



Phases in IR system

1. Retrieval and Ranking process
2. Indexing process
3. Crawling process

IR Applications

1. Web Search Engines
2. Digital Libraries & Archives
3. Social Media Search & Monitoring
4. Medical Information Retrieval
5. Multimedia Retrieval
6. Question Answering Systems
7. Recommendation & Discovery Systems

Query processing

- Queries are formal statements of information needs
- Types of Queries:

Type	Description	Example
Keyword Query	Just a set of words; no structure.	<i>"best pizza recipes"</i>
Boolean Query	Uses logical operators (AND, OR, NOT).	<i>"apple AND banana NOT orange"</i>
Phrase Query	Exact sequence of words (in quotes).	<i>"machine learning algorithms"</i>
Proximity Query	Finds terms close to each other.	<i>"solar NEAR/3 energy"</i>
Wildcard Query	Uses special symbols to represent one or more unknown characters	<i>"hist* AND "ancient Rome"</i>
Natural Language Query	Full sentences or questions.	<i>"What are the top AI companies in 2025?"</i>

Basic IR system phases

1. Pre-processing
 - a. Tokenization
 - b. Stop-word removal*
 - c. Stemming*
 - d. Normalization*
2. Index Construction
 - a. Generate corpus from central repository
 - b. Create and update Document-Term /Term-Document Incidence matrix
3. Query processing
 - a. Extract matching columns (in document-term matrix) / rows (in term-document matrix) with each query terms from incidence matrix
 - b. Produce results based on the truth values

Example IR Problem

- **Grepping:** simplest form of document retrieval is for a computer to do this sort of linear scan through documents.
- We need more efficient way:
 - To process large document collections quickly
 - To allow more flexible matching operations
 - To allow ranked retrieval
- **Indexing:** The way to avoid linearly scanning the texts for each query is to index the documents in advance.

Example

	Central Repository Doc1 = “SRM University AP” Doc2 = “AP is State”	User Queries Q1 = “AP” Q2 = “SRM AP”
Pre-processing	1. Tokenization Doc1 → [‘srm’, ‘university’, ‘ap’] → 3 terms Doc2 → [‘ap’, ‘is’, ‘state’] → 3 terms Q1 → [‘ap’] → 1 term Q2 → [‘srm’, ‘ap’] → 2 terms	

Example ...

	Central Repository Doc1 = “SRM University AP” Doc2 = “AP is State”	User Queries Q1 = “AP” Q2 = “SRM AP”																		
Index Construction (Forward indexing)	1. Corpus [‘srm’, ‘university’, ‘ap’, ‘is’, ‘state’] →5 terms	2. Document-term matrix <table><tr><td></td><td>srm</td><td>university</td><td>ap</td><td>is</td><td>state</td></tr><tr><td>Doc1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>Doc2</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr></table>		srm	university	ap	is	state	Doc1	1	1	1	0	0	Doc2	0	0	1	1	1
	srm	university	ap	is	state															
Doc1	1	1	1	0	0															
Doc2	0	0	1	1	1															

Example ...

	Central Repository Doc1 = “SRM University AP” Doc2 = “AP is State”	User Queries Q1 = “AP” Q2 = “SRM AP”															
Query Execution	1. Query terms extraction Q1: <table><tr><td></td><td>ap</td></tr><tr><td>Doc1</td><td>1</td></tr><tr><td>Doc2</td><td>1</td></tr></table> Q2: <table><tr><td></td><td>srm</td><td>ap</td></tr><tr><td>Doc1</td><td>1</td><td>1</td></tr><tr><td>Doc2</td><td>0</td><td>1</td></tr></table>		ap	Doc1	1	Doc2	1		srm	ap	Doc1	1	1	Doc2	0	1	2. Relevant documents retrieval Q1 → [‘Doc1’, ‘Doc2’] Q2 → [‘Doc1’]
	ap																
Doc1	1																
Doc2	1																
	srm	ap															
Doc1	1	1															
Doc2	0	1															

Problem with Document-term matrix

1. Search inefficiency
2. Difficulty in querying
3. Limited support for Ranking

Solution: Inverted indexing / Term-document incidence matrix

	Doc1	Doc2
srm	1	0
university	1	0
ap	1	1
is	0	1
state	0	1

Example IR Problem with Inverted indexing

Shakespeare's Collected Works

- Used about 32,000 different words

Query1: Brutus AND Calpurnia

Binary term-document incidence matrix

- Terms are the indexed units

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

Boolean Retrieval Model

- It is a model poses any query which is in the form of a **Boolean expression of terms**.
- **Terms** are combined with the operators **AND, OR, and NOT**.
- Views each **document** as just a set of words.
- **Documents** means whatever units we have decided to build a retrieval system over.
- **Corpus**: group of documents over which we perform retrieval.

Term	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0

Boolean Query Processing

A conjunctive query: It is defined as a type of query that consists of multiple subgoals or conjuncts.

Query1: Brutus AND Calpurnia

Term	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Brutus	1	1	0	1	0	0
Calpurnia	0	1	0	0	0	0

Boolean Query Processing

Query2: Brutus AND Caesar AND Calpurnia

Term	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0

Query3: Brutus AND Caesar AND NOT Calpurnia
???

Problem with term-document matrix

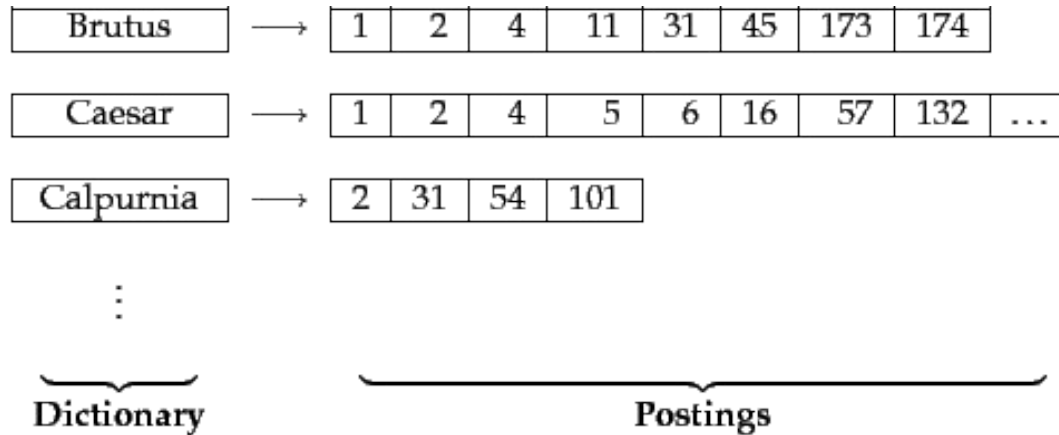
Disadvantages of term-document matrix:

- Adding new document is difficult: need to add new columns
- For a large collection with fewer words for document
 - For example term-document matrix of 550K X 1M
 - Matrix is extremely sparse

Solution:

- Keep a dictionary of terms
- Postings or posting list: list that records which documents the term occurs in

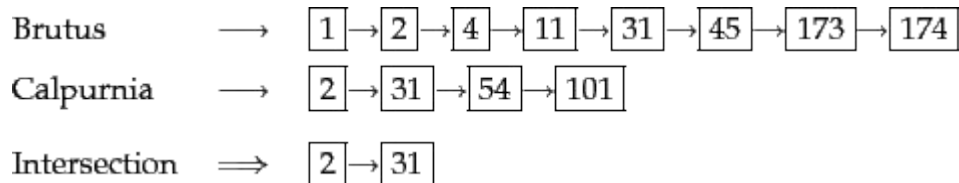
Inverted index with postings



Problem: Query processing?

Processing Boolean queries

Query1: Brutus AND Calpurnia



Query2: Brutus AND Caesar AND Calpurnia ?

Query3: Brutus AND Caesar AND NOT Calpurnia ?

Processing Boolean queries

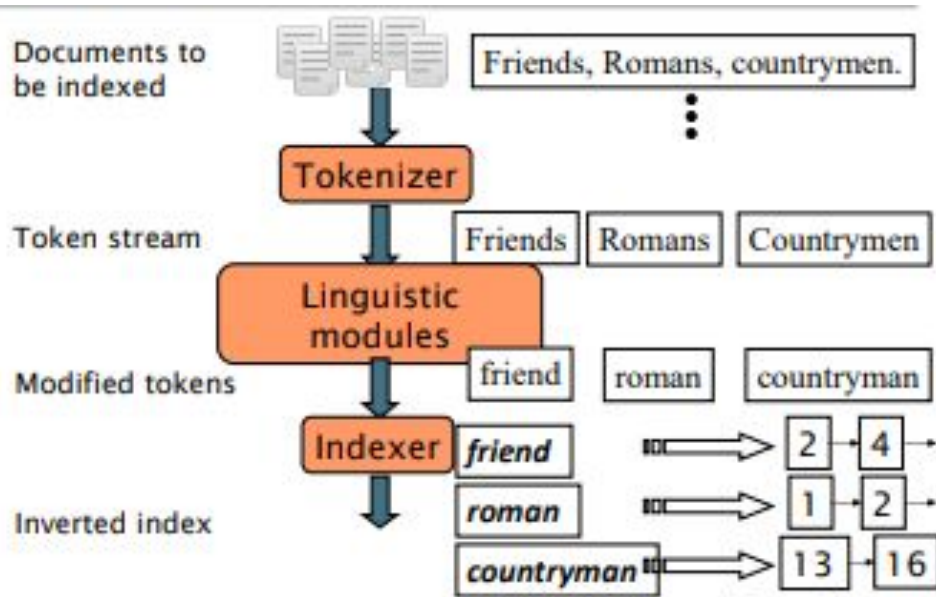
Intersection of two postings lists p_1 and p_2

```
INTERSECT( $p_1, p_2$ )  
  1   $answer \leftarrow \langle \rangle$   
  2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$   
  3  do if  $docID(p_1) = docID(p_2)$   
  4      then  $\text{ADD}(answer, docID(p_1))$   
  5           $p_1 \leftarrow next(p_1)$   
  6           $p_2 \leftarrow next(p_2)$   
  7  else if  $docID(p_1) < docID(p_2)$   
  8      then  $p_1 \leftarrow next(p_1)$   
  9      else  $p_2 \leftarrow next(p_2)$   
 10 return  $answer$ 
```

Time complexity: $O(m+n)$

Index construction

- **Index construction or Indexing:** the process or machine that performs it the indexer.
- The indexer needs **raw text**, but documents are **encoded** in many ways.
- Indexers **compress and decompress** intermediate files and the final index.



Major steps in Inverted Index construction:

1. **Collect the documents** to be indexed.
2. **Tokenize** the text.
3. Do **linguistic preprocessing** of tokens.
4. **Index the documents** that each term occurs in.

- Token stream generation:
 - The first step of processing is to convert this byte sequence into a **linear sequence of characters**.

استقلت الجزائر في سنة 1962 بعد 132 عاما من الاحتلال الفرنسي.

← → ← →

← START

‘Algeria achieved its independence in 1962 after 132 years of French occupation.’

- Determine what the **document unit** for indexing is?
- **Tokenization** is the **process of chopping** character streams into **tokens**.

Input: Friends, Romans, Countrymen, lend me your ears;

Output:

Friends	Romans	Countrymen	lend	me	your	ears
---------	--------	------------	------	----	------	------

- **Token** is an instance of a sequence of characters in some particular document that are grouped together **as a useful semantic unit** for processing.

Tokenization ...

- **Type** is the class of all tokens containing the same character sequence.
- **Term** is a type that is included in the IR system's dictionary.
- What are the correct tokens to use?

*Mr. **O'Neill** thinks that the boys' stories about Chile's capital **aren't** amusing.*

For *O'Neill*, which of the following is the desired tokenization? And for *aren't*, is it:

neill
oneill
o'neill
o' neill
o neill?

aren't
arent
are n't
aren t,

- These issues of tokenization are language-specific

Stop words

- **Stop words:** some extremely common words which would appear to be of little value in helping select documents matching a user need are excluded from the vocabulary entirely.
- The general strategy for determining a stop list:
 - Sort the terms by collection frequency.
 - Take the most frequent terms (often hand-filtered for their semantic content)
- A stop list of 25 semantically non-selective words which are common in Reuters-RCV1.

a an and are as at be by for from
has he in is it its of on that the
to was were will with

Stemming and lemmatization

- Documents are going to use
 - different forms of a word
 - families of derivationally related words with similar meanings
- Goal is to reduce inflectional forms and sometimes derivationally related forms of a word to a **common base form**.

am, are, is $\Rightarrow$ be
car, cars, car's, cars' $\Rightarrow$ car

- **Stemming** usually refers to a crude heuristic process that chops off the ends of words in the hope of achieving this goal correctly most of the time
 - often includes the removal of derivational affixes.
 - stemming most commonly collapses derivationally related words
- **Lemmatization** usually refers to doing things properly with the use of a vocabulary and morphological analysis of words.
 - normally aiming to remove inflectional endings only
 - Lemma: the base or dictionary form of a word
 - lemmatization commonly only collapses the different inflectional forms of a lemma

Common stemming algorithms

1. **Porter Stemmer:** Developed by Martin Porter in 1980, it is one of the most widely used and well-known stemming algorithms. It applies a series of heuristic rules to iteratively remove common suffixes from English words, such as "-ing," "-ed," and "-es," to reduce them to their root form. While simple and efficient, the resulting stem may not always be a meaningful word.
2. **Snowball Stemmer (Porter2):** An extension of the Porter Stemmer, also developed by Martin Porter. It offers improved accuracy and supports multiple languages beyond English. It is considered more robust than the original Porter Stemmer and is also known as Porter2.
3. **Lancaster Stemmer:** This algorithm is known for being more aggressive than the Porter Stemmer, often leading to shorter stems. It employs a set of rules for suffix removal or replacement, and some rules can be applied iteratively. It might sometimes "over-stem" words, resulting in stems that are not easily recognizable.
4. **Lovins Stemmer:** Proposed in 1968, it was one of the earliest published stemming algorithms. It works by removing the longest suffix from a word and then potentially recoding the stem to form a valid word.

Porter's algorithm

Rule

SSSES → SS

IES → I

SS → SS

S →

Example

caresses → caress

ponies → poni

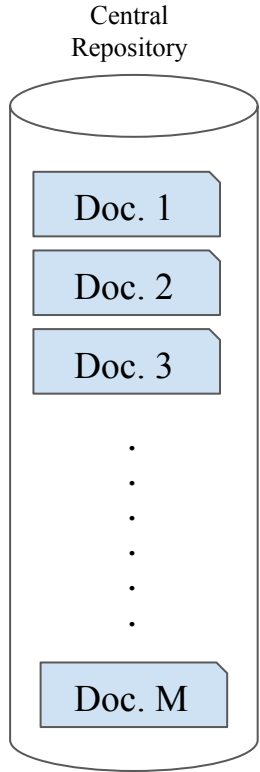
caress → caress

cats → cat

Basic steps of Indexer:

- **First pass: compile the vocabulary**
 - **Collection** assembling all term-docID pairs.
- **Second pass: construct the inverted index**
 - **Sort** the pairs with the term as the dominant key and docID as the secondary key.
 - **Organize postings list:** the docIDs for each term into a postings list and compute statistics like term and document frequency.

Sort-based indexing (SBI) ...



termID-docID
dictionary

termID	docID
t1	1
t2	1
.	.
t9	1
t32	1
t1	2
.	.
t32	2
.	.
t59	2
t62	2
.	.
t9	M
t32	M
.	.
t59	M
t62	M

Sort
dictionary

termID	docID
t1	1
t1	2
t2	1
.	.
t9	1
t9	M
.	.
t32	1
t32	2
t32	M
.	.
t59	2
t59	M
.	.
t62	2
t62	M
.	.

termID-docID
Posting List

termID	docID Posting List
t1 [2]	1->2
t2 [1]	1
.	.
t9 [2]	1->M
.	.
t32 [3]	1->2->M
.	.
t59 [2]	2->M
.	.
t62 [2]	2->M
.	.

Sort-based Index Construction

Doc 1

I did enact Julius Caesar: I was killed
i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus
hath told you Caesar was ambitious:

term	docID	term	docID
I	1	ambitious	2
did	1	be	2
enact	1	brutus	1
julius	1	brutus	2
caesar	1	capitol	1
I	1	caesar	1
was	1	caesar	2
killed	1	caesar	2
i'	1	did	1
the	1	enact	1
capitol	1	hath	1
brutus	1	I	1
killed	1	I	1
me	1	i'	1
so	2	it	2
let	2	julius	1
it	2	killed	1
be	2	killed	1
with	2	let	2
caesar	2	me	1
the	2	noble	2
noble	2	so	2
brutus	2	the	1
hath	2	the	2
told	2	told	2
you	2	you	2
caesar	2	was	1
was	2	was	2
ambitious	2	with	2

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

- For small collections, all this can be done in memory.

Problem:

- What about for large collection of documents?

Boolean Retrieval Exercises

Exercises.

- Draw the inverted index that would be built for the following document collection. (See Figure [1.3](#) for an example.)

Doc 1 new home sales top forecasts

Doc 2 home sales rise in july

Doc 3 increase in home sales in july

Doc 4 july new home sales rise

- Consider these documents:

Doc 1 breakthrough drug for schizophrenia

Doc 2 new schizophrenia drug

Doc 3 new approach for treatment of schizophrenia

Doc 4 new hopes for schizophrenia patients

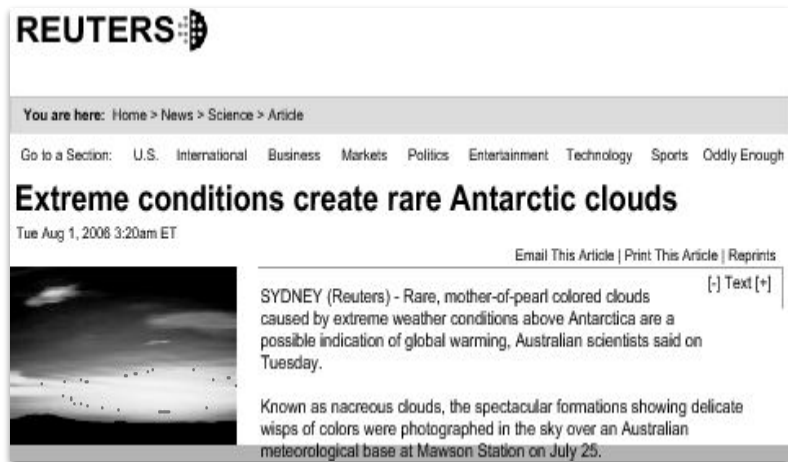
1. Draw the term-document incidence matrix for this document collection.
 2. Draw the inverted index representation for this collection, as in Figure [1.3](#) (page .
- For the document collection shown in Exercise [1.2](#) , what are the returned results for these queries:
 1. schizophrenia AND drug
 2. for AND NOT(drug OR approach)

Index construction ...

Collection statistics for Reuters-RCV1

```
>>> from sklearn.datasets import fetch_rcv1
```

```
>>> rcv1 = fetch_rcv1()
```



Symbol	Statistic	Value
N	documents	800,000
L_{ave}	avg. # tokens per document	200
M	terms	400,000
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
	avg. # bytes per term	7.5
T	tokens	100,000,000

Sort-based Index Construction - Problems

- As we build the index, we parse docs one at a time.
 - The final postings for any term are incomplete until the end.
- At 8 bytes per (termID, docID), demands a lot of space for large collections.
- $T = 100,000,000$ in the case of RCV1
 - So ... we can do this in memory today, but typical collections are much larger. E.g., the New York Times provides an index of >150 years of newswire
- **Thus: We need to store intermediate results on disk.**

Sort using disk as “memory”?

- Can we use the same index construction algorithm for larger collections, but by using disk instead of memory?
- No: Sorting $T = 100,000,000$ records on disk is too slow – too many disk seeks.
- We need an **external sorting** algorithm.

The design of **indexing algorithms** is governed by **hardware constraints**.

- **Caching:** Access to data in memory is much faster than access to data on disk.
- **Disk seek time:** To maximize data transfer rates, chunks of data that will be read together should therefore be stored contiguously on disk.
- **Buffer:** Reading a single byte from disk can take as much time as reading the entire block.
- **Disk I/O is block-based:** Reading and writing of entire blocks (as opposed to smaller chunks).
- **Disk space** is several orders of magnitude larger.
- **Fault tolerance is very expensive:** it is much cheaper to use many regular machines rather than one fault tolerance systems

Typical system parameters in 2007

Symbol	Statistic	Value
s	average seek time	5 ms = 5×10^{-3} s
b	transfer time per byte	$0.02 \mu\text{s} = 2 \times 10^{-8}$ s
	processor's clock rate	10^9 s^{-1}
p	lowlevel operation	
	(e.g., compare & swap a word)	$0.01 \mu\text{s} = 10^{-8}$ s
	size of main memory	several GB
	size of disk space	1 TB or more

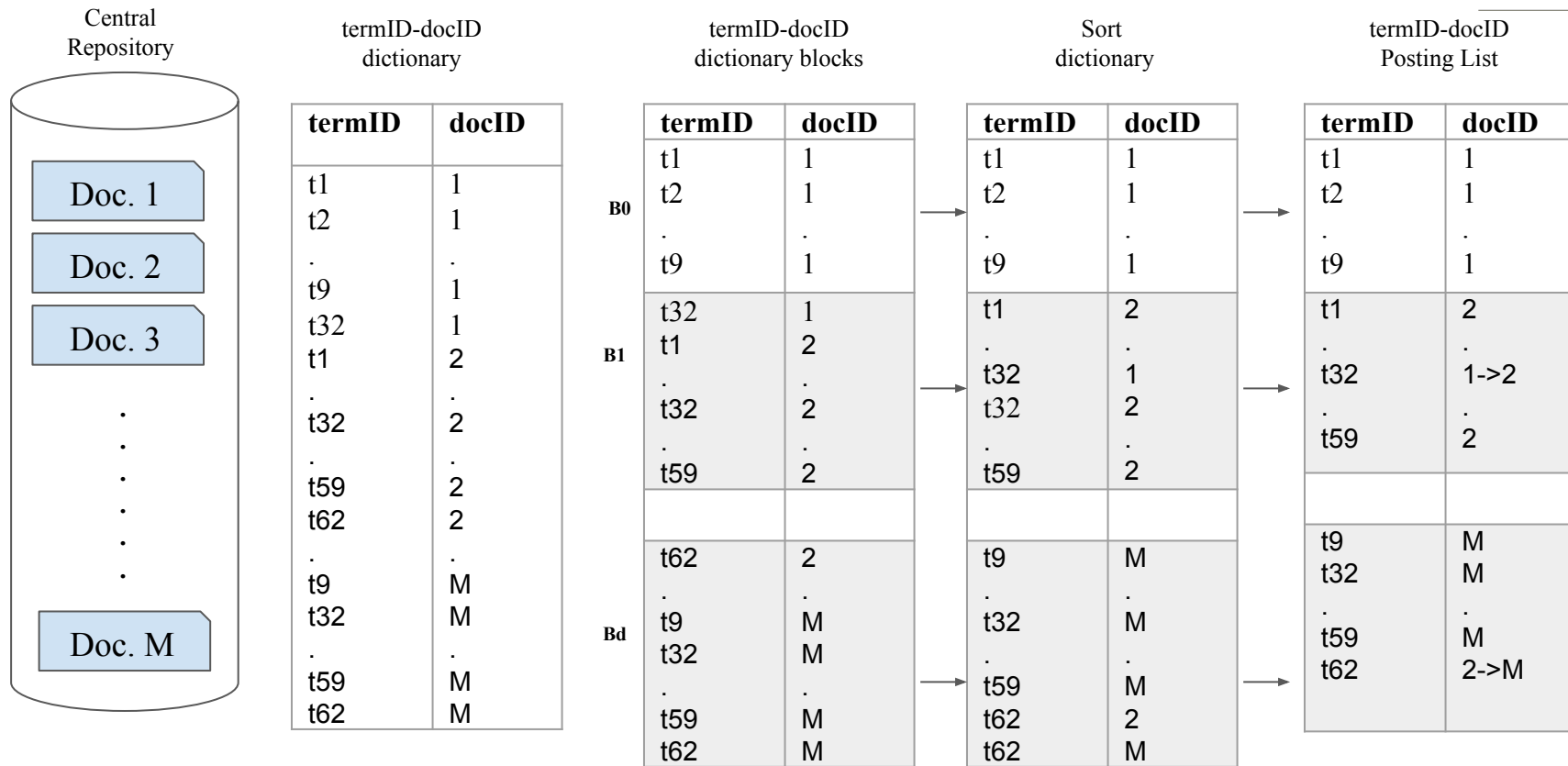
Types of indexing

- Blocked sort-based indexing (BSBI)
- Single-pass in-memory indexing (SPIMI)
- Distributed indexing
- Dynamic indexing

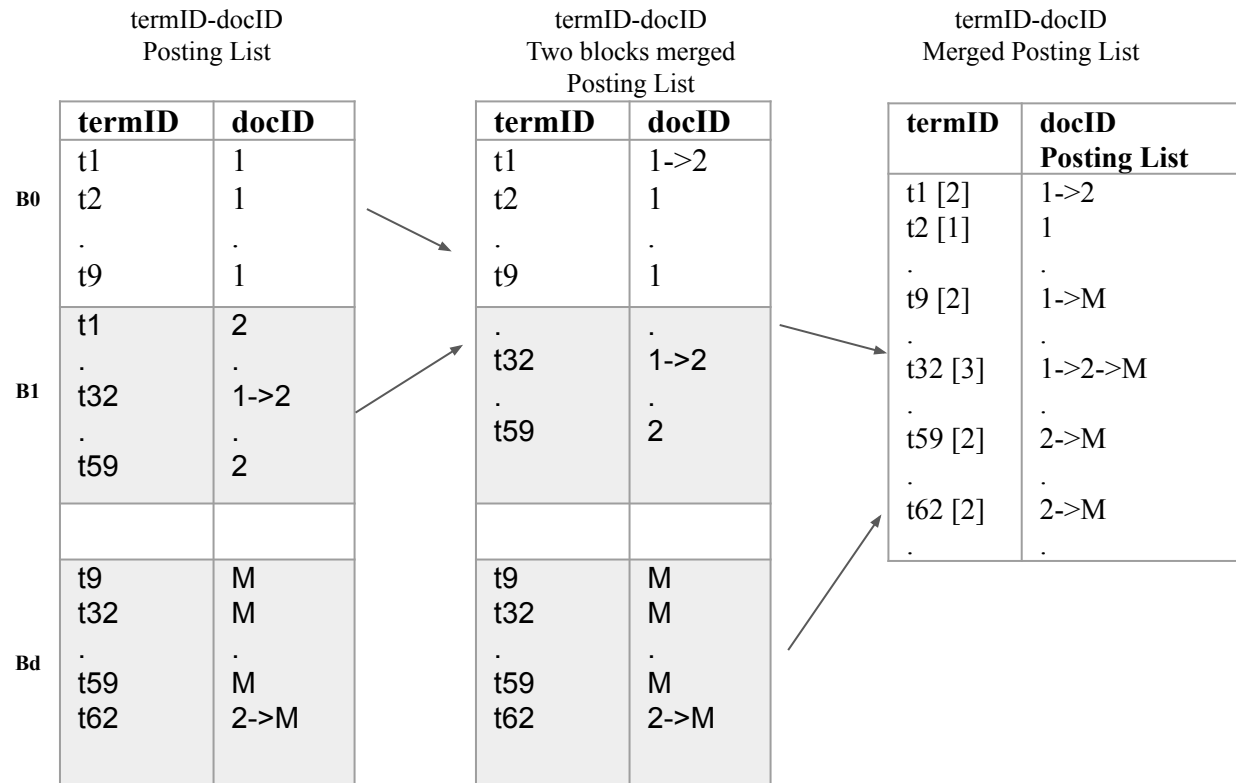
Blocked sort-based indexing (BSBI)

- 8-byte records (termID, docID)
- These are generated as we parse docs
- Must now sort 100M such 8-byte records by termID
- Define a Block $\sim$ 10M such records
 - Can easily fit a couple into memory
 - Will have 10 such blocks to start with
- Basic idea of algorithm:
 - Accumulate postings for each block, sort, write to disk
 - Then merge the blocks into one long sorted order

Blocked sort-based indexing (BSBI)



Blocked sort-based indexing (BSBI)

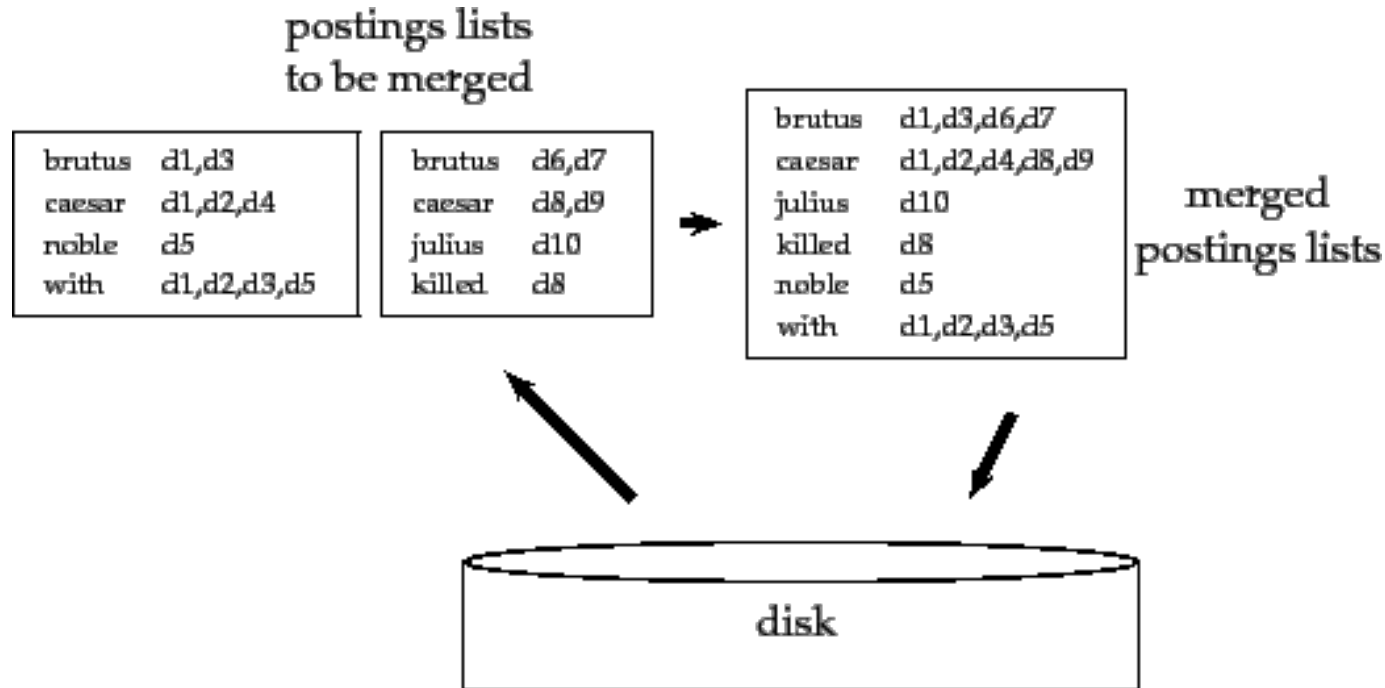


Blocked sort-based indexing (BSBI)

Sorting 10 blocks of 10M records

- First, read each block and sort within:
 - Quicksort takes $O(N \ln N)$ expected steps
 - In our case $N=10M$
- 10 times this estimate – gives us 10 sorted runs of 10M records each.
- Done straightforwardly, need 2 copies of data on disk
 - But can optimize this?
- Can do binary merges, with a merge tree of $\log_2 10 = 4$ layers.
- During each layer, read into memory runs in blocks of 10M, merge, write back.

Blocked sort-based indexing (BSBI)



Blocked sort-based indexing (BSBI) ...

BSBIINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5       $\text{BSBI-INVERT}(block)$ 
6       $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{merged})$ 
```

How expensive is BSBI?

Its time complexity is $\Theta(T \log T)$ because the step with the highest time complexity is sorting and T is an upper bound for the number of items we must sort

Blocked sort-based indexing (BSBI) ...

- But it is more efficient to do a **multi-way merge**, where you are reading from all blocks simultaneously
 - Open all block files simultaneously and maintain a read buffer for each one and a write buffer for the output file
 - In each iteration, pick the lowest termID that hasn't been processed using a priority queue
 - Merge all postings lists for that termID and write it out
- To reduce disk seeks: read decent-sized chunks of each block into memory and then write out a decent-sized output chunk

Problem with BSBI

- Our assumption was: we can keep the dictionary in memory.
- We need the dictionary (which grows dynamically) in order to implement a term to termID mapping.
- Intermediate files become very large.

Single-pass in-memory indexing (SPIMI)

- Key idea 1: Generate separate dictionaries for each block – no need to maintain term-termID mapping across blocks.
- Key idea 2: Don't sort. Accumulate postings in postings lists as they occur.
- With these two ideas we can generate a complete inverted index for each block.
- These separate indexes can then be merged into one big index.

Single-pass in-memory indexing (SPIMI)

```
SPIMI-INVERT(token_stream)
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token  $\leftarrow$  next(token_stream)
5      if term(token)  $\notin$  dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8          if full(postings_list)
9              then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10         ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms  $\leftarrow$  SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

The time complexity of SPIMI is $\Theta(T)$ because no sorting of tokens is required and all operations are at most linear in the size of the collection.

Exercises

1. Brutus AND Calpurnia
2. (Brutus OR Caesar) AND NOT Calpurnia
3. Brutus AND Caesar AND Calpurnia
4. (Calpurnia AND Brutus) AND Caesar
5. (madding OR crowd) AND (ignoble OR strife) AND (killed OR stain)

What about distributed systems?

Web search engine data centers

- Web search data centers (Google, Bing, Baidu) mainly contain commodity machines.
- Data centers are distributed around the world.
- Estimate: Google ~1 million servers, 3 million processors/cores (Gartner 2007)

Massive data centers

- If in a non-fault-tolerant system with 1000 nodes, each node has 99.9% uptime, what is the uptime of the entire system?
- Answer: 37% - meaning, 63% of the time one or more servers is down.
- Exercise: Calculate the number of servers failing per minute for an installation of 1 million servers.

Distributed indexing ...

- Web search engines use **distributed indexing** algorithms for index construction.
- **Distributed index** that is partitioned across several machines - either according to term or according to document.
- Distributed indexing for a **term-partitioned index** is an application of **MapReduce**.
- **MapReduce** is a general architecture for distributed computing.
 - Maintain a **master machine directing the indexing job** – considered “safe”.
 - Break up indexing into **sets of (parallel) tasks**: Parsers, Inverters
 - **Master machine** assigns each task to an idle machine from a pool.

- **Parser:**

- Master assigns a split to an idle parser machine
- Parser reads a document at a time and emits (term, doc) pairs
- Parser writes pairs into j partitions;
 - Example: Each partition is for a range of terms' first letters (e.g., a-f, g-p, q-z) – here $j = 3$.

- **Inverter:**

- An inverter collects all (term,doc) pairs (= postings) for one term-partition.
- Sorts and writes to postings lists

- **Index construction:**

- Index construction was just one phase.
- Another phase: transforming a term-partitioned index into a document-partitioned index.
 - Term-partitioned: one machine handles a subrange of terms
 - Document-partitioned: one machine handles a subrange of documents

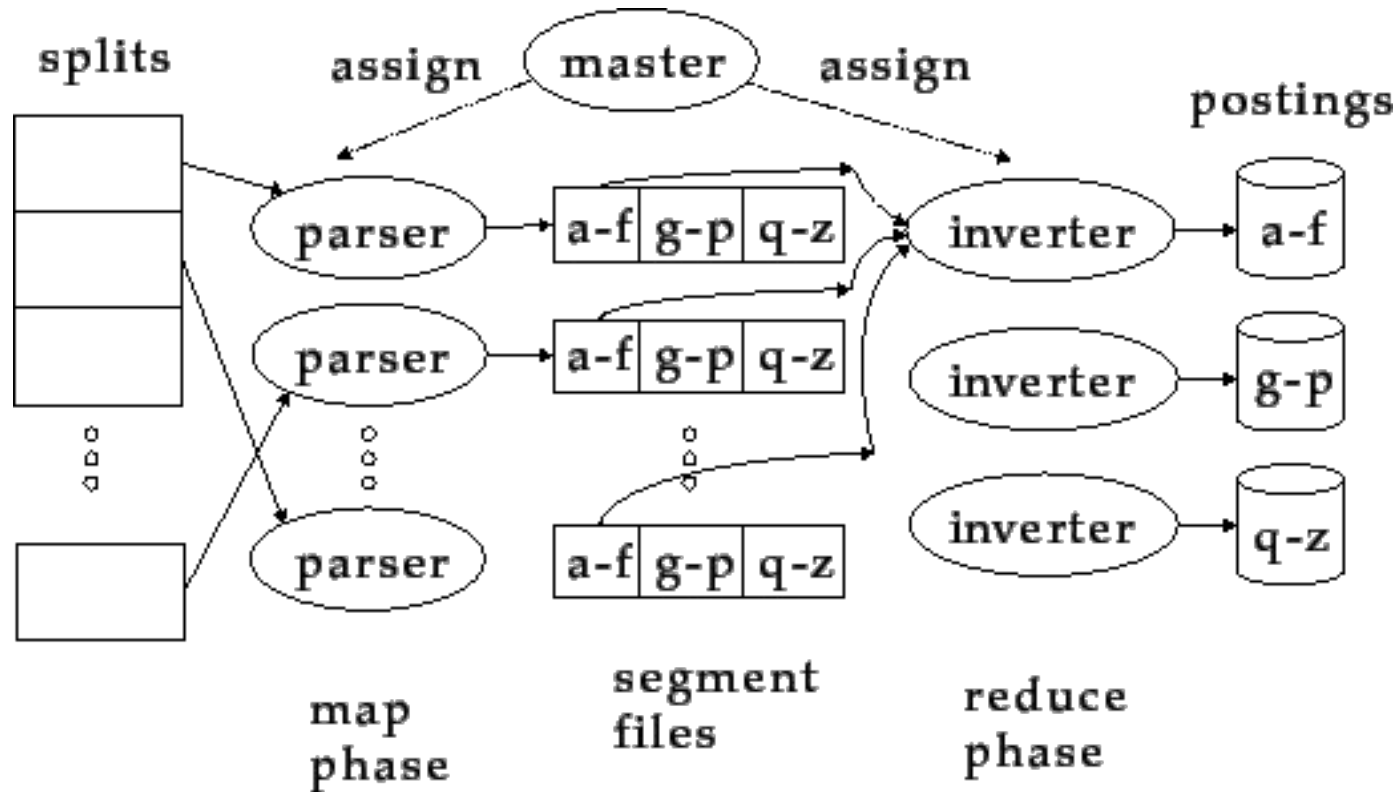
- **Mapreduce:**

- The index construction algorithm is an instance of MapReduce.
- MapReduce is a robust and conceptually simple framework for distributed computing.
- Google indexing system as consisting of a number of phases, each implemented in MapReduce.

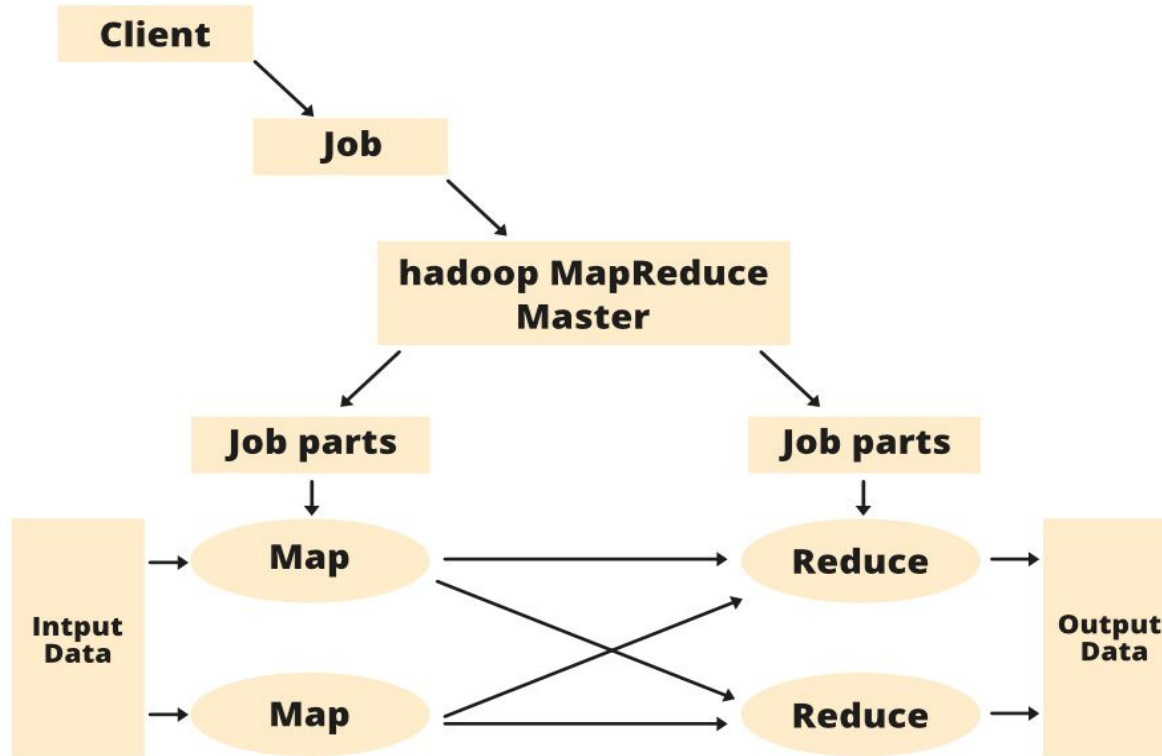
Distributed indexing ...

- **MapReduce** breaks a large computing problem into smaller parts by recasting it in terms of manipulation of **key-value pairs**.
- Steps in MapReduce:
 - The map phase of MapReduce consists of mapping splits of the input data to key-value pairs.
 - For the reduce phase , we want all values for a given key to be stored close together, so that they can be read and processed quickly.
 - Collecting all values (here: docIDs) for a given key (here: termID) into one list is the task of the inverters in the reduce phase.
 - The master assigns each term partition to a different inverter - and, as in the case of parsers, reassigns term partitions in case of failing or slow inverters.
 - Finally, the list of values is sorted for each key and written to the final sorted postings list (“postings” in the figure).

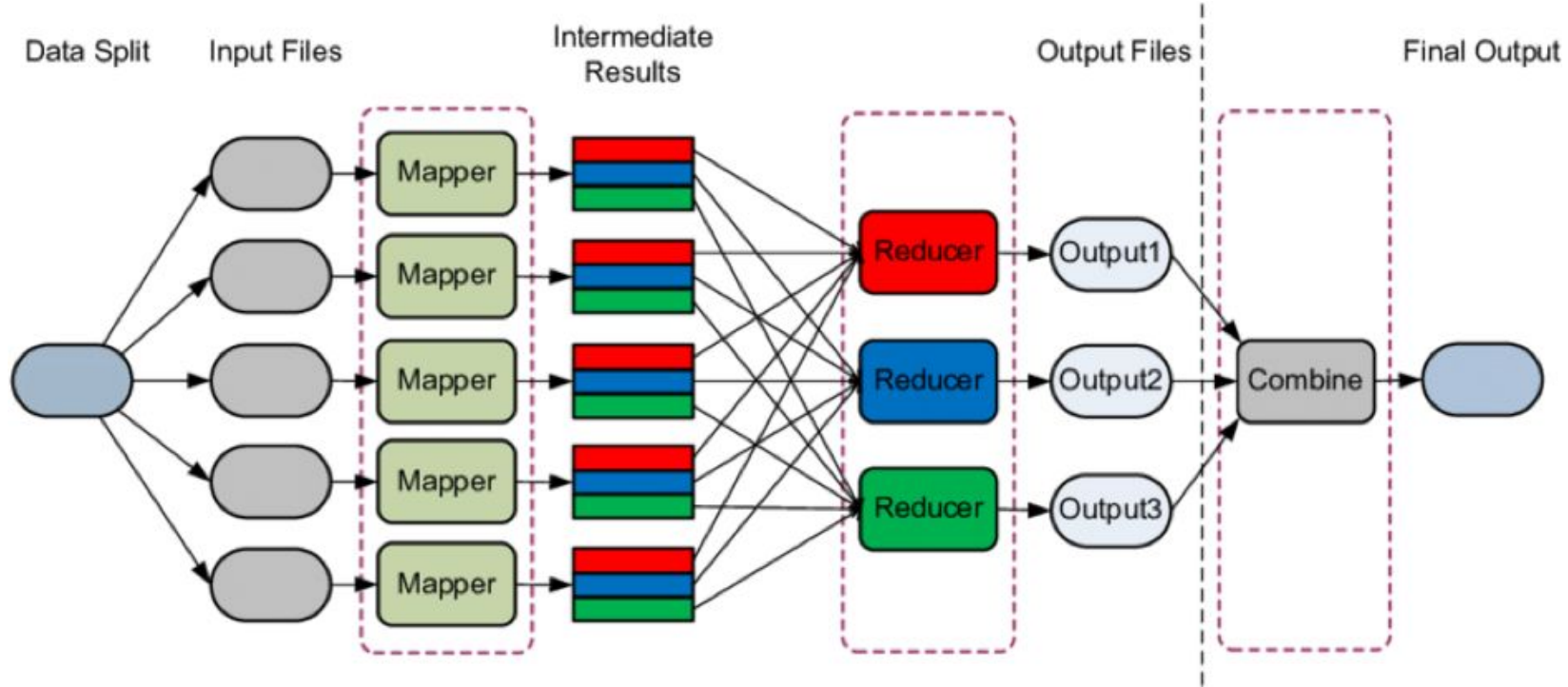
Distributed Indexing



Mapreduce architecture



How mapreduce work



Mapreduce example

Schema of map and reduce functions

map: input

→ list(k, v)

reduce: ($k, \text{list}(v)$)

→ output

Instantiation of the schema for index construction

map: web collection

→ list(termID, docID)

reduce: ($\langle \text{termID}_1, \text{list}(\text{docID}) \rangle, \langle \text{termID}_2, \text{list}(\text{docID}) \rangle, \dots$) → (postings_list1, postings_list2, ...)

Example for index construction

map: $d_2 : \text{C died}, d_1 : \text{C came}, \text{C c'ed}.$

→ ($\langle \text{C}, d_2 \rangle, \langle \text{died}, d_2 \rangle, \langle \text{C}, d_1 \rangle, \langle \text{came}, d_1 \rangle, \langle \text{C}, d_1 \rangle, \langle \text{c'ed}, d_1 \rangle$)

reduce: ($\langle \text{C}, (d_2, d_1, d_1) \rangle, \langle \text{died}, (d_2) \rangle, \langle \text{came}, (d_1) \rangle, \langle \text{c'ed}, (d_1) \rangle$)

→ ($\langle \text{C}, (d_1:2, d_2:1) \rangle, \langle \text{died}, (d_2:1) \rangle, \langle \text{came}, (d_1:1) \rangle, \langle \text{c'ed}, (d_1:1) \rangle$)

- Till now the assumption is the **document collection is static**.
- What about if **document collection is dynamic**?
 - The simplest way to achieve this is to **periodically reconstruct the index** from scratch.
- If there is a requirement that new documents be included quickly?
- Maintain two indexes: **a large main index and a small auxiliary index** that stores new documents.
 - The auxiliary index is kept in memory.
 - Searches are run across both indexes and results merged.
 - Deletions are stored in an invalidation bit vector.
 - We can then filter out deleted documents before returning the search result.
 - Documents are updated by deleting and reinserting them.
 - Each time the auxiliary index becomes too large, we merge it into the main index.
- The reason for keeping the auxiliary index is **to reduce the number of disk seeks** required over time.

Issues with main and auxiliary indexes

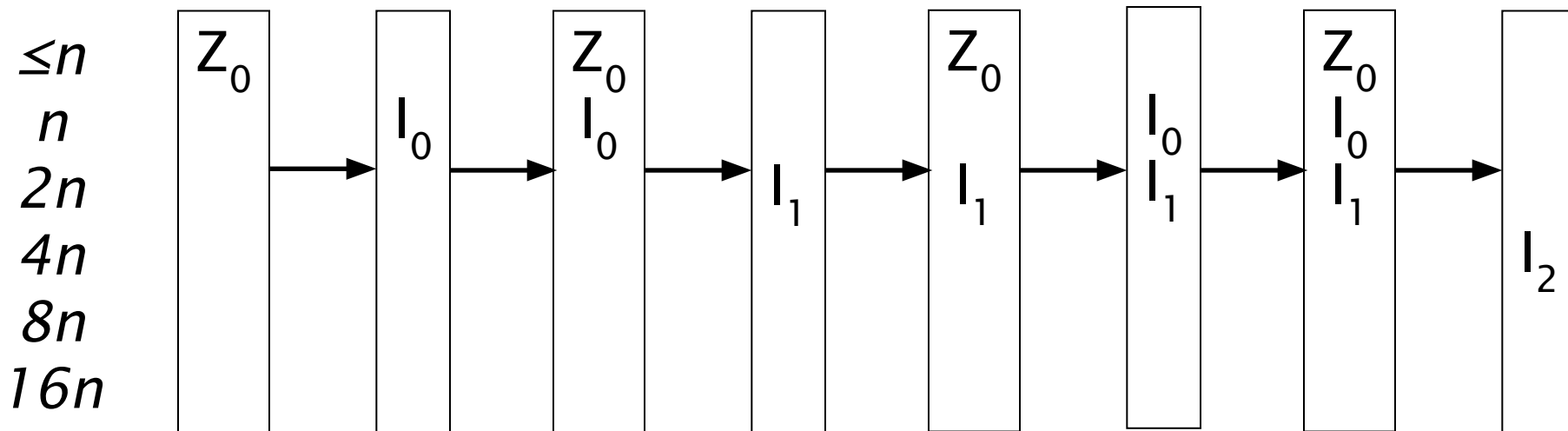
- Problem of frequent merges : poor performance during merge
- Actually:
 - Merging of the auxiliary index into the main index is efficient if we keep a separate file for each postings list.
 - Merge is the same as a simple append.
 - But then we would need a lot of files – inefficient for OS.
- Assumption:
 - The index is one big file.
- In reality:
 - Use a scheme somewhere in between (e.g., split very large postings lists, collect postings lists of length 1 in one file etc.)

Logarithmic merge

- Maintain a series of indexes, each twice as large as the previous one
 - At any time, some of these powers of 2 are instantiated
- Keep smallest (Z_0) in memory
- Larger ones ($I_0, I_1, \dots$) on disk
- If Z_0 gets too big ($> n$), write to disk as I_0
- or merge with I_0 (if I_0 already exists) as Z_1
- Either write merge Z_1 to disk as I_1 (if no I_1)
- Or merge with I_1 to form Z_2

Dynamic indexing ...

Logarithmic merge in action



Dynamic Indexing ...

```
LMERGEADDTOKEN(indexes,  $Z_0$ , token)
1   $Z_0 \leftarrow \text{MERGE}(Z_0, \{\text{token}\})$ 
2  if  $|Z_0| = n$ 
3    then for  $i \leftarrow 0$  to  $\infty$ 
4      do if  $I_i \in \text{indexes}$ 
5        then  $Z_{i+1} \leftarrow \text{MERGE}(I_i, Z_i)$ 
6           ( $Z_{i+1}$  is a temporary index on disk.)
7            $\text{indexes} \leftarrow \text{indexes} - \{I_i\}$ 
8        else  $I_i \leftarrow Z_i$  ( $Z_i$  becomes the permanent index  $I_i$ .)
9            $\text{indexes} \leftarrow \text{indexes} \cup \{I_i\}$ 
10         BREAK
11      $Z_0 \leftarrow \emptyset$ 
```

```
LOGARITHMICMERGE()
1   $Z_0 \leftarrow \emptyset$  ( $Z_0$  is the in-memory index.)
2   $\text{indexes} \leftarrow \emptyset$ 
3  while true
4  do LMERGEADDTOKEN(indexes,  $Z_0$ , GETNEXTTOKEN())
```

Dynamic indexing ...

- Auxiliary and main index:
 - T/n merges where T is # of postings and n is size of auxiliary
 - Index construction time is $O(T^2/n)$ as in the worst case a posting is touched T/n times
- Logarithmic merge:
 - Each posting is merged at most $O(\log (T/n))$ times, so complexity is $O(T \log (T/n))$
 - So logarithmic merge is much more efficient for index construction
- But query processing now requires the merging of $O(\log (T/n))$ indexes
 - Whereas it is $O(1)$ if you just have a main and auxiliary index

Query Processing in details

- Query: Brutus AND Caesar AND Calpurnia
- Assumption:
 - Brutus appears in 10 documents.
 - Caesar appears in 5 documents.
 - Calpurnia appears in 3 documents.
- How many comparisons?
 - Option 1: Merge Brutus AND Caesar first. Merge the result with Calpurnia.
 - Option 2: Merge Caesar AND Calpurnia first. Merge the result with Brutus.

Query Processing in details ...

- Option 1: Merge Brutus AND Caesar first. Merge the result with Calpurnia.
 - Brutus AND Caesar: In worst case, requires $(10 + 5) = 15$ comparisons.
 - Result AND Calpurnia: In worst case, requires $(5 + 3) = 8$ comparisons.
 - Therefore, requires $15 + 8 = 23$ comparisons.
- Option 2: Merge Caesar AND Calpurnia first. Merge the result with Brutus.
 - Caesar AND Calpurnia: In worst case, requires $5 + 3 = 8$ comparisons.
 - Result AND Brutus: In worst case, requires $3 + 10 = 13$ comparisons.
 - Therefore, $13 + 8 = 21$ comparisons.

Query Processing in details ...

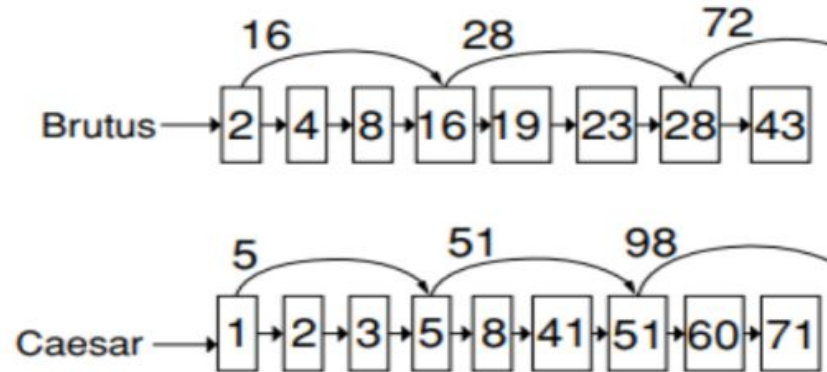
Term	Postings size
eyes	213312
kaleidoscope	87009
marmalade	107913
skies	271658
tangerine	46653
trees	316812

What about (tangerine OR trees) AND (marmalade OR skies) AND
(kaleidoscope OR eyes)?

Answer: ((kaleidoscope OR eyes) AND (tangerine OR trees)) AND (marmalade or skies)

Faster postings list intersection via skip pointers

- If the list lengths are m and n , the **intersection** takes $O(m+n)$
- Can we do better than this? **Skip list**
- **Skip list**: augmenting postings lists with skip pointers (at indexing time)
- **Skip pointers** are effectively shortcuts that allow us to avoid processing parts of the postings list that will not figure in the search results.



Intersection with skip lists

INTERSECTWITHSKIPS(p_1, p_2)

```
1  answer  $\leftarrow \{ \}$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then ADD(answer,  $\text{docID}(p_1)$ )
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then if  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
9          then while  $\text{hasSkip}(p_1)$  and  $(\text{docID}(\text{skip}(p_1)) \leq \text{docID}(p_2))$ 
10             do  $p_1 \leftarrow \text{skip}(p_1)$ 
11             else  $p_1 \leftarrow \text{next}(p_1)$ 
12  else if  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
13      then while  $\text{hasSkip}(p_2)$  and  $(\text{docID}(\text{skip}(p_2)) \leq \text{docID}(p_1))$ 
14          do  $p_2 \leftarrow \text{skip}(p_2)$ 
15          else  $p_2 \leftarrow \text{next}(p_2)$ 
16  return answer
```

Faster postings list intersection via skip pointers

- Skip pointers only helps for AND queries, not for OR queries
- Where do we place skips?
 - **More skips means** shorter skip spans, lots of comparisons to skip pointers.
 - **Fewer skips means** long skip spans, few pointer comparisons.
 - **Simple heuristic for placing skips:** use $\sqrt{P}$ evenly-spaced skip pointers for postings list of length P . —
- Building effective skip pointers is **easy** if an index is relatively **static**.

Skip lists

- Advantages:
 - Achieves sublinear list intersection time which is better than $O(m+n)$.
 - Works better for static index.
 - Useful for AND queries.
- Disadvantages:
 - Needs space to store skip pointers.
 - The effectiveness depends on where we place skip pointers
 - More skip: shorted skip interval and more comparisons
 - Less skip: Less extra space more linear traversal and lesser comparison

Phrase queries and positional postings

- Many complex or technical concepts and many organization and product names are **multiword compounds or phrases**.
- Most recent search engines **support a double quotes syntax** (“stanford university”) for phrase queries.
- As many as **10% of web queries are phrase queries**.
- **Postings lists** to be simply lists of documents that contain individual terms **are not sufficient**.
- Approaches for handling phrase queries:
 - Bi-word indexes
 - Positional indexes

- Consider every **pair of consecutive terms** in a document as a phrase.

For example: *Friends, Romans, Countrymen*

Biwords: *friends romans* | *romans countrymen*

- **Nouns and noun phrases** have a special status in describing the concepts people are interested in searching for.
- **Steps for bi-wards generation:**
 - First, we tokenize the text and perform part-of-speech-tagging.
 - We can then group terms into nouns, including proper nouns, (N) and function words, including articles and prepositions, (X), among other classes.
 - Now deem any string of terms of the form NX*N to be an extended biword.
 - Each such extended biword is made a term in the vocabulary.

- Example: *cost overruns on a power plant*

“cost overruns” AND “overruns power” AND “power plant”

- Can be extended to longer sequences of words, and if the index includes variable length word sequences.
- Searches for a single term are not naturally handled in a biword index.
- **Disadvantages:**
 - So we also need to have an index of single-word terms
 - Always a chance of false positive matches
 - Exhaustive biword dictionary greatly expands the size of the vocabulary

Positional indexes

← “to” appears 993K times overall.

to, 993427:

⟨ 1, 6: ⟨7, 18, 33, 72, 86, 231⟩; ← “to” appears six times in doc 1
2, 5: ⟨1, 17, 74, 222, 255⟩; at positions 7, 18,
4, 5: ⟨8, 16, 190, 429, 433⟩;
5, 2: ⟨363, 367⟩;
7, 3: ⟨13, 23, 191⟩; ... ⟩

be, 178239:

⟨ 1, 2: ⟨17, 25⟩;
4, 5: ⟨17, 191, 291, 430, 434⟩;
5, 3: ⟨14, 19, 101⟩; ... ⟩

Positional indexes ...

• Query: “ $to_1 be_2 or_3 not_4 to_5 be_6$ ”

• TO, 993427:

- $\langle 1: \langle 7, 18, 33, 72, 86, 231 \rangle;$
- $2: \langle 1, 17, 74, 222, 255 \rangle;$
- $4: \langle 8, 16, 190, 429, 433 \rangle;$
- $5: \langle 363, 367 \rangle;$
- $7: \langle 13, 23, 191 \rangle; \dots \rangle$

• BE, 178239:

- $\langle 1: \langle 17, 25 \rangle;$
- $4: \langle 17, 191, 291, 430, 434 \rangle;$
- $5: \langle 14, 19, 101 \rangle; \dots \rangle$

• Document 4 is a match!

Positional index size:

- Requires postings storage significantly.
- Expect a positional index to be 2 to 4 times as large as a non-positional index.
- Complexity of a Boolean query is $\theta(T)$ rather than $\theta(N)$.
- Where, T is total number of tokens in the document collection.

Proximity search:

- Example:
 - employment /3 place
 - / k within k words of (on either side)
- Positional indexes can be used for such queries, biword indexes cannot.

```
POSITIONALINTERSECT( $p_1, p_2, k$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $l \leftarrow \langle \rangle$ 
5           $pp_1 \leftarrow \text{positions}(p_1)$ 
6           $pp_2 \leftarrow \text{positions}(p_2)$ 
7          while  $pp_1 \neq \text{NIL}$ 
8              do while  $pp_2 \neq \text{NIL}$ 
9                  do if  $|\text{pos}(pp_1) - \text{pos}(pp_2)| \leq k$ 
10                     then  $\text{ADD}(l, \text{pos}(pp_2))$ 
11                     else if  $\text{pos}(pp_2) > \text{pos}(pp_1)$ 
12                         then break
13                      $pp_2 \leftarrow \text{next}(pp_2)$ 
14                     while  $l \neq \langle \rangle$  and  $|l[0] - \text{pos}(pp_1)| > k$ 
15                         do  $\text{DELETE}(l[0])$ 
16                     for each  $ps \in l$ 
17                         do  $\text{ADD}(answer, \langle \text{docID}(p_1), \text{pos}(pp_1), ps \rangle)$ 
18                      $pp_1 \leftarrow \text{next}(pp_1)$ 
19                  $p_1 \leftarrow \text{next}(p_1)$ 
20                  $p_2 \leftarrow \text{next}(p_2)$ 
21             else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
22                 then  $p_1 \leftarrow \text{next}(p_1)$ 
23             else  $p_2 \leftarrow \text{next}(p_2)$ 
24  return  $answer$ 
```

Combination scheme

- biword indexes and positional indexes can be fruitfully combined.
- A combination strategy uses
 - a phrase index/biword index, for certain queries
 - a positional index for other phrase queries.

- Compression is required when we need **less disk space**.
- Advantages of compression:
 - Increased use of caching.
 - Faster transfer of data from disk to memory
- If the main goal of compression is to conserve disk space, then the **speed of compression algorithms is of no concern**.
- But for improved cache utilization and faster disk-to-memory transfer, **decompression speeds must be high**.
- **Better compression ratios** can be achieved with **lossy compression**, which discards some information.
- **Case folding, stemming, and stop word elimination** are forms of lossy compression.

Lossless vs. lossy compression

- **Lossless compression:** All information is preserved.
 - What we mostly do in IR.
- **Lossy compression:** Discard some information
 - Several of the preprocessing steps can be viewed as lossy compression: case folding, stop words, stemming, number elimination.
 - Prune postings entries that are unlikely to turn up in the top k list for any query. Almost no loss of quality in top k list.

Why compression for inverted indexes?

- Dictionary
 - Make it small enough to keep in main memory
 - Make it so small that you can keep some postings lists in main memory too
- Postings file(s)
 - Reduce disk space needed
 - Decrease time needed to read postings lists from disk
 - Large search engines keep a significant part of the postings in memory.
 - Compression lets you keep more in memory
- We will devise various IR-specific compression schemes

Statistical properties of terms in IR

size of	word types (terms)			non-positional postings			positional postings		
	dictionary			non-positional index			positional index		
	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %
Unfiltered	484			109,971			197,879		
No numbers	474	-2	-2	100,680	-8	-8	179,158	-9	-9
Case folding	392	-17	-19	96,969	-3	-12	179,158	0	-9
30 stopwords	391	-0	-19	83,390	-14	-24	121,858	-31	-38
150 stopwords	391	-0	-19	67,002	-30	-39	94,517	-47	-52
stemming	322	-17	-33	63,812	-4	-42	94,517	0	-52

Statistical properties of terms in IR ...

Vocabulary size vs. collection size

- How big is the term vocabulary?
 - That is, how many distinct words are there?
- Can we assume an upper bound?
 - Not really: At least $7020 = 1037$ different words of length 20
- In practice, the vocabulary will keep growing with the collection size
 - Especially with Unicode

Important laws:

- Heaps' law: Estimating the number of terms
- Zipf's law: Modeling the distribution of terms

Heaps' law

- Heaps' law estimates **vocabulary size as a function of collection size**.
- Vocabulary size **M** as a function of collection size **T** (number of tokens).

$$M = kT^b$$

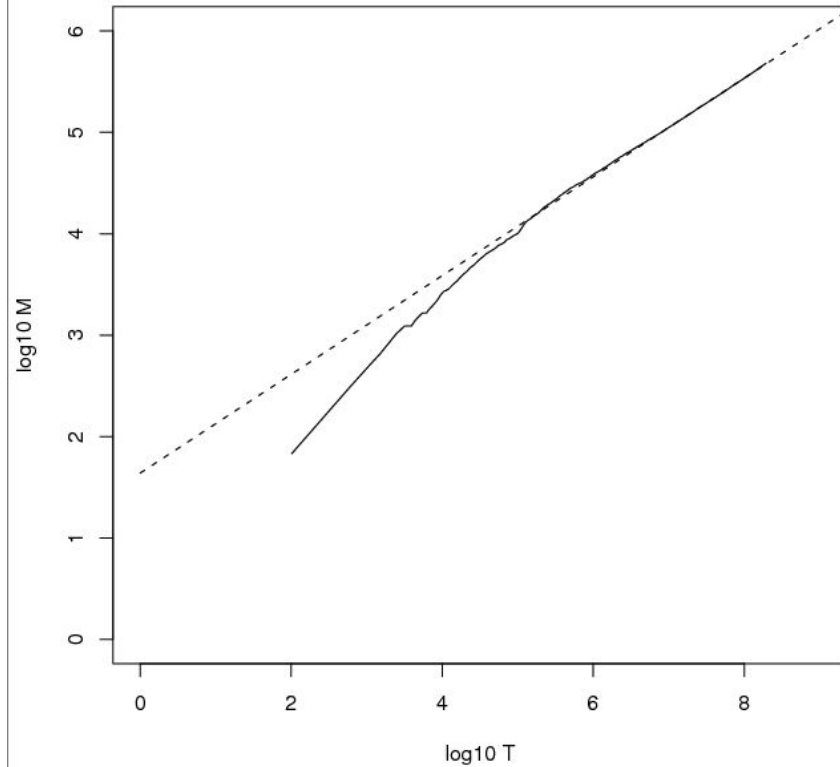
- **M** is the size of the vocabulary, **T** is the number of tokens in the collection
- Typical values: $30 \leq k \leq 100$ and $b \approx 0.5$
- In a log-log plot of vocabulary size **M** vs. **T**, Heaps' law predicts a line with slope about $\frac{1}{2}$
- It is the simplest possible (linear) relationship between the two in log-log space

$$\log M = \log k + b \log T$$

- An empirical finding (“empirical law”).
- **Motivation for Heaps' law:** simplest possible relationship between collection size and vocabulary size is **linear in log-log space**.

Heaps' law for Reuters RCV1

- For RCV1, the dashed line $\log_{10} M = 0.49 \log_{10} T + 1.64$ is the best least squares fit.
- Thus, $k \approx 44$ and $b \approx 0.49$.
- Good empirical fit for Reuters RCV1 !
- For first 1,000,020 tokens, law predicts 38,323 terms; actually, 38,365 terms



Zipf's law

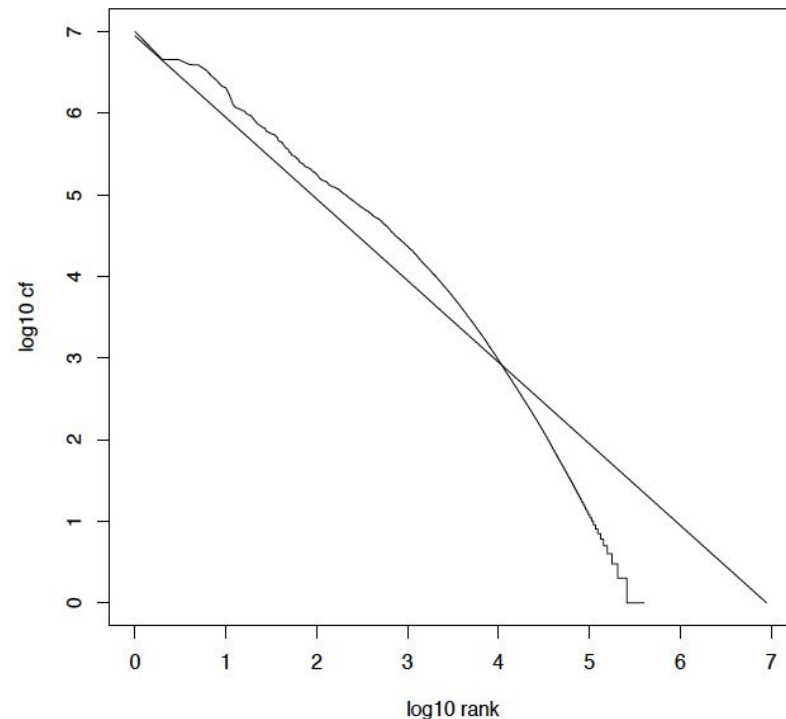
- Heaps' law gives the vocabulary size in collections.
- We also study the relative frequencies of terms.
- In natural language, there are a few very frequent terms and very many very rare terms.
- **Zipf's law:** The i th most frequent term has frequency proportional to $1/i$.

$$cf_i \propto 1/i = K/i$$

- where K is a normalizing constant, cf_i is collection frequency: the number of occurrences of the term t_i in the collection.
- If the most frequent term (the) occurs cf_1 times
 - then the second most frequent term (of) occurs $cf_1/2$ times
 - the third most frequent term (and) occurs $cf_1/3$ times ...

Zipf's law for Reuters RCV1

- Equivalent: $cf_i = K/i$ where K is a normalizing factor, so
- $\log cf_i = \log K - \log i$
- Linear relationship between $\log cf_i$ and $\log i$
- Another power law relationship



Visualizing search interface

1. Visualizing boolean syntax → VQuery
2. Visualizing query terms within retrieval results → TileBars
3. Visualizing relationships among words and documents → VIBE
4. Visualization of text mining → WordTee

Dictionary compression

- Main goal of compressing the dictionary is to **fit it in main memory**.
- Memory footprint competition with other applications.
- Embedded/mobile devices may have very little memory
- Even if the dictionary isn't in memory, we want it to be small for a fast search startup time
- Methods for dictionary compression:
 - Dictionary as fixed-width string
 - Dictionary as term pointers into string
 - Dictionary as blocked storage
 - Dictionary as blocked storage and front coding

Dictionary as fixed-width string

- Simplest data structure for the dictionary is to sort the vocabulary lexicographically and **store it in an array of fixed-width entries**

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→

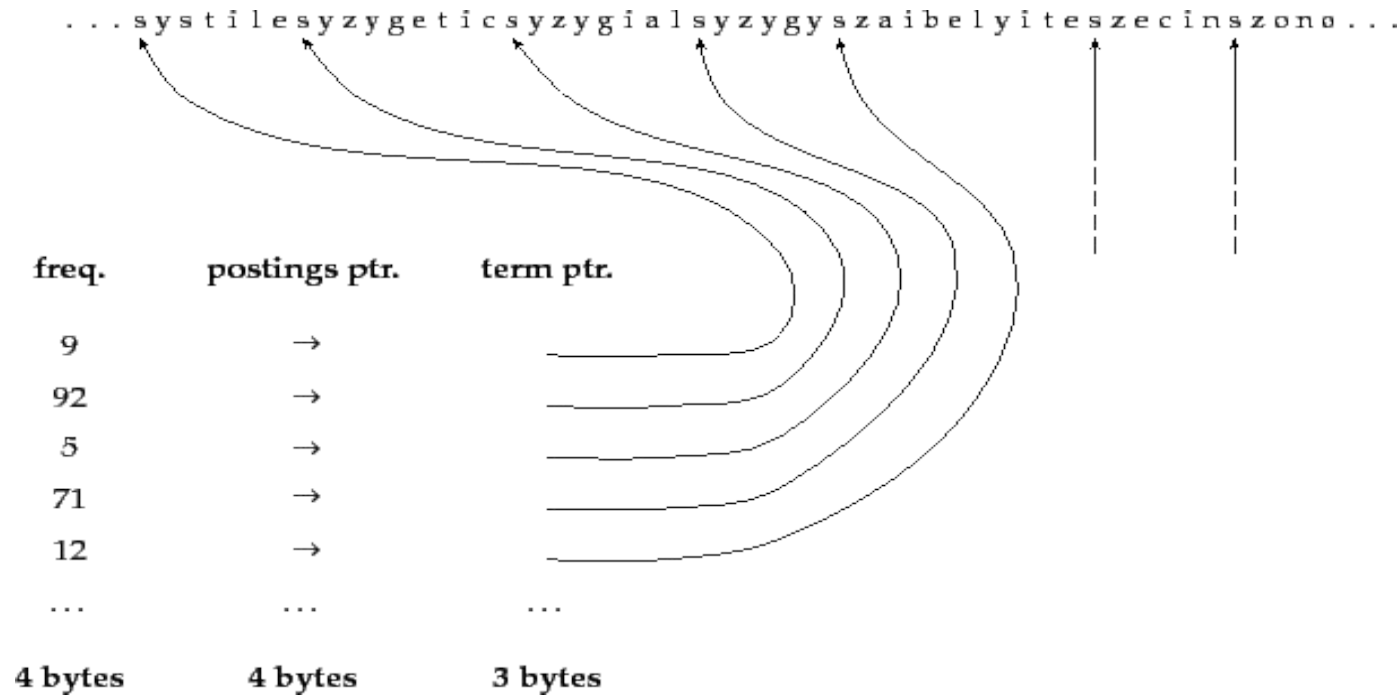
space needed: 20 bytes 4 bytes 4 bytes

- For Reuters-RCV1: $M \times (20 + 4 + 4) = 400000 \times 28 = 11.2$ MB for storing the dictionary in this scheme.
- Pointers mark the end of the preceding term and the beginning of the next.
- For example, the first three terms in this example are systile, syzygetic, and syzygial.

Dictionary as term pointers into string

- **Disadvantage of fixed-width entries:**
 - Using fixed-width entries for terms is clearly wasteful.
 - The average length of a term in English is about 8 characters.
 - Wasting twelve characters (or 24 bytes) in the fixed-width scheme.
 - We have no way of storing terms with more than twenty characters.
- A **pointer to the next term** is also used to demarcate the **end of the current term**.
- We locate terms in the data structure by way of **binary search in the** (now smaller) table.
- This scheme **saves us 60% compared** to fixed-width storage - 24 bytes on average of the 40 bytes 12 bytes on average of the 20 bytes we allocated for terms before.
- We now also need to store term pointers.

Dictionary as term pointers into string

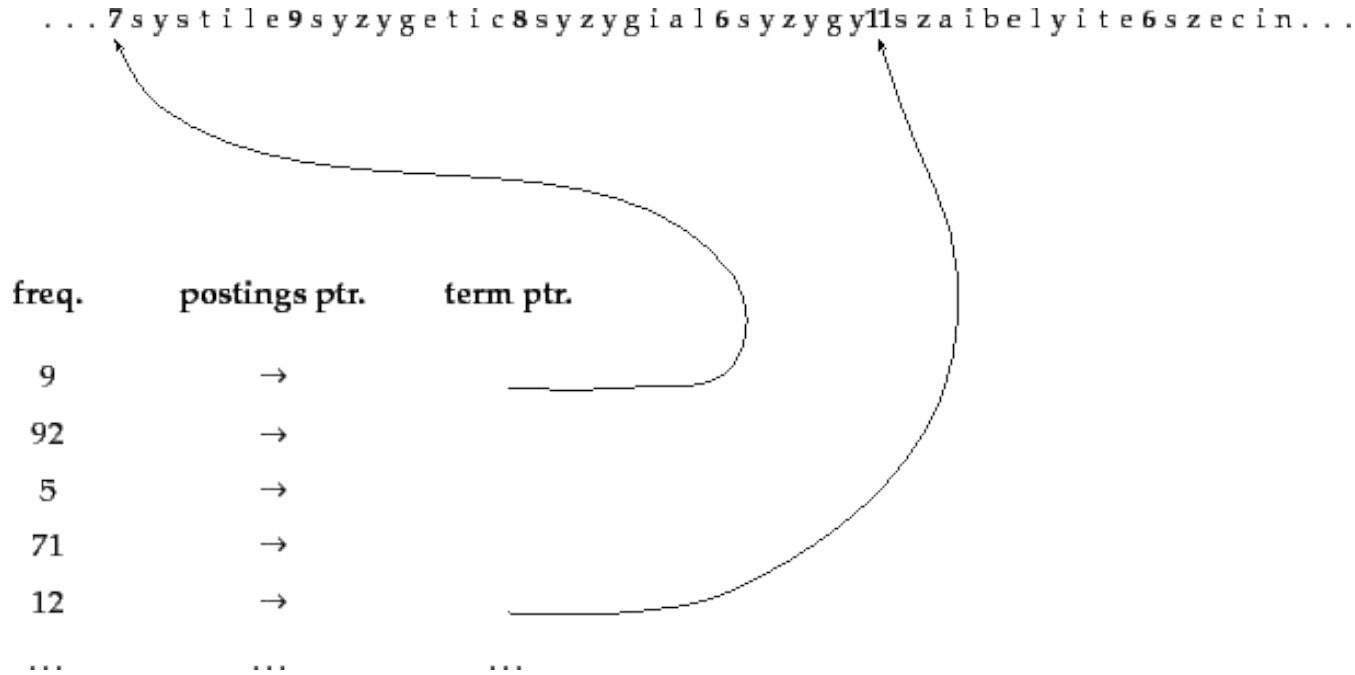


- In this new scheme, we need $400000 \times (4 + 4 + 3 + 8) = 7.6$ MB for the Reuters-RCV1 dictionary.

Dictionary as blocked storage

- We can further compress the dictionary **by grouping terms in the string into of size k** and keeping a term pointer only for the first term of each block.
- Each term is preceded by a byte encoding its length that indicates how many bytes to skip to reach subsequent terms.
- We store the length of the term in the string as an additional byte at the beginning of the term.
- We thus eliminate $k-1$ term pointers, but need an additional k bytes for storing the length of each term.
- For $k=4$, we save $(k-1) \times 3 = 9$ bytes for term pointers, but need an additional $k=4$ bytes for term lengths.

Dictionary as blocked storage



- So the total space requirements for the dictionary of Reuters-RCV1 are reduced by 5 bytes per four-term block, or a total of $400000 \times 1/4 \times 5 = 0.5$ MB.

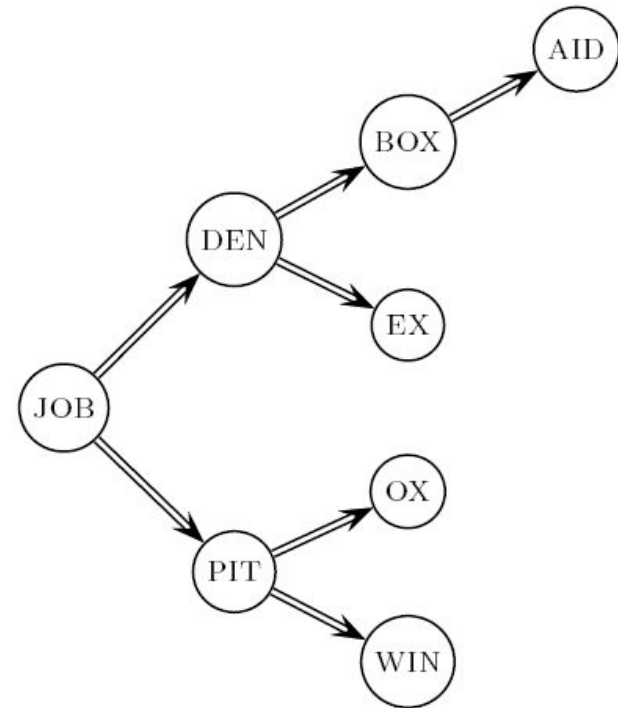
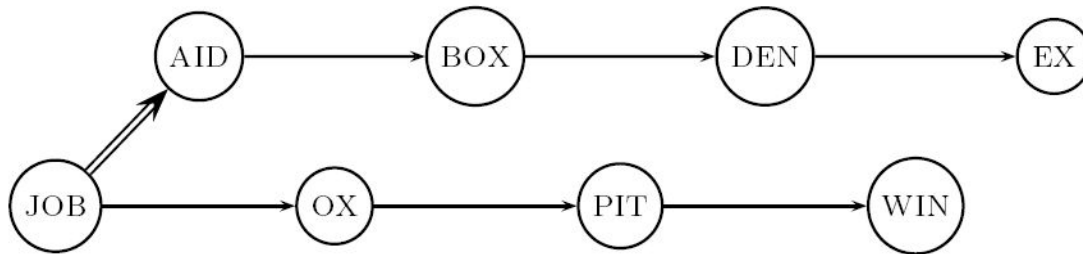
Dictionary search

Dictionary search with blocking

- Assuming each dictionary term equally likely in query, average number of comparisons = $(0+1+2+3+2+1+2+2)/8 = 1.6$

Dictionary search without blocking

- Binary search down to 4-term block
- Then linear search through terms in block.
- Blocks of 4 (binary tree), average comparisons = $(0+1+2+3+4+1+2+3)/8 = 2$



Dictionary as blocked storage and front coding

- **Front coding:** One source of redundancy in the dictionary we have not exploited yet is the fact that consecutive entries in an alphabetically sorted list share common prefixes.
- A common prefix is identified for a subsequence of the term list and then referred to with a special character.

One block in blocked compression ($k = 4$) ...
8automata8automate9automatic10automation



...further compressed with front coding.
8automat*a1◊e2◊ic3◊ion

Dictionary compression for Reuters-RCV1

	data structure	size in MB	
	dictionary, fixed-width	19.211.2	
	dictionary, term pointers into string	10.8 7.6	
	$\sim$, with blocking, $k = 4$	10.3 7.1	
	$\sim$, with blocking & front coding	7.9 5.9	

Other methods: Hashing, B-Trees

Recall from Reuters-RCV1

- It has 800,000 documents, 200 tokens per document, six characters per token, and 100,000,000 postings.
- Document identifiers are $\log_2 800,000 = 20$ bits
- Size of the collection is about $800,000 \times 200 \times 6 \text{ bytes} = 960 \text{ MB}$
- Size of the uncompressed postings file is $100,000,000 \times 20 / 8 = 250 \text{ MB}$
- We can observe that the postings for **frequent terms are close together**.
- **Key idea:** gaps between postings are short, requiring a lot less space than 20 bits to store.
- Gaps for the **most frequent terms are mostly equal to 1**.

Postings file compression ...

		encoding	postings list										
	the	docIDs	...		283042		283043		283044		283045	...	
		gaps				1		1		1		...	
	computer	docIDs	...		283047		283154		283159		283202	...	
		gaps				107		5		43		...	
	arachnocentric	docIDs	252000		500100								
		gaps	252000	248100									

Gaps are encoded using Variable Byte (VB) encoding.

Variable byte (VB) encoding

- It uses an integral number of bytes to encode a gap.
- The last 7 bits of a byte are “payload” and encode part of the gap.
- The first bit of the byte is a continuation bit . It is set to 1 for the last byte of the encoded gap and to 0 otherwise.
- To decode a variable byte code, we read a sequence of bytes with continuation bit 0 terminated by a byte with continuation bit 1.
- We then extract and concatenate the 7-bit parts.

docIDs	824	829	215406
gaps	-	5	214577
VB code	00000110 10111000	10000101	00001101 00001100 10110001

Variable byte encoding ...

VBENCODENUMBER(*n*)

```
1  bytes ← {}
2  while true
3  do PREPEND(bytes, n mod 128)
4    if n < 128
5      then BREAK
6    n ← n div 128
7  bytes[LENGTH(bytes)] += 128
8  return bytes
```

VBENCODE(*numbers*)

```
1  bytestream ← {}
2  for each n ∈ numbers
3  do bytes ← VBENCODENUMBER(n)
4    bytestream ← EXTEND(bytestream, bytes)
5  return bytestream
```

VBDECODE(*bytestream*)

```
1  numbers ← {}
2  n ← 0
3  for i ← 1 to LENGTH(bytestream)
4  do if bytestream[i] < 128
5    then n ← 128 × n + bytestream[i]
6    else n ← 128 × n + (bytestream[i] - 128)
7    APPEND(numbers, n)
8    n ← 0
9  return numbers
```

Variable byte (VB) encoding

- With VB compression, the size of the compressed index for Reuters-RCV1 is 116 MB as we verified in an experiment.
- The idea of VB encoding can also be applied to larger or smaller units than bytes: 32-bit words, 16-bit words, and 4-bit words or nibbles .
- Word sizes smaller than bytes get even better compression ratios at the cost of more bit manipulation.
- For most IR systems variable byte codes offer an excellent tradeoff between time and space.
- We can achieve better compression ratios by using **bit-level** encodings, in particular two closely related encodings: Gamma codes, and Delta codes

- VB codes use an adaptive number of bytes depending on the size of the gap.
- Bit-level codes adapt the length of the code on the finer grained bit level.
- The simplest bit-level code is **unary code**.
- The unary code of n is a string of n 1s followed by a 0.
- How efficient can a code be in principle?
 - Assuming the 2^n gaps G with $1 \leq G \leq 2^n$ are all equally likely, the optimal encoding uses n bits for each G .
 - So some gaps ($G=2^n$ in this case) cannot be encoded with fewer than $\log_2 G$ bits.
 - Our goal is to get as close to this lower bound as possible.

Gamma codes ...

	number	unary code	length	offset	γ code	
	0	0				
	1	10	0		0	
	2	110	10	0	10,0	
	3	1110	10	1	10,1	
	4	11110	110	00	110,00	
	9	111111110	1110	001	1110,001	
	13		1110	101	1110,101	
	24		11110	1000	11110,1000	
	511		111111110	11111111	111111110,11111111	
	1025		11111111110	0000000001	11111111110,0000000001	

Gamma codes ...

- A method that is within a factor of optimal is **Gamma** encoding .
- **Gamma codes** implement variable-length encoding by splitting the representation of a gap G into a pair of length and offset.
- Offset is G in binary, but with the leading 1 removed.
- Example:
 - For 13 (binary 1101) offset is 101.
 - Length encodes the length of offset in unary code.
 - For 13, the length of offset is 3 bits, which is 1110 in unary.
- The Gamma code of 13 is therefore 1110101, the concatenation of length 1110 and offset 101.

Gamma codes ...

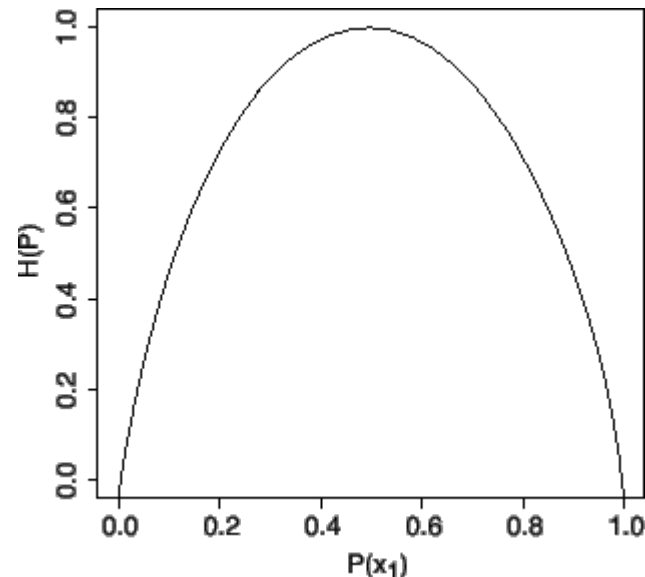
- A Gamma code is decoded by first reading the unary code up to the 0 that terminates it.
 - For example, the four bits 1110 when decoding 1110101.
 - Now we know how long the offset is: 3 bits.
 - The offset 101 can then be read correctly and the 1 that was chopped off in encoding is prepended: $101 \rightarrow 1101 = 13$.
- The length of offset is $\lfloor \log_2 G \rfloor$ bits and the length of length is $\lfloor \log_2 G \rfloor + 1$, so the length of the entire code is $2 \times \lfloor \log_2 G \rfloor + 1$ bits.
- Gamma codes are always of odd length and they are within a factor of 2 of what we claimed to be the optimal encoding length $\log_2 G$.
- We derived this optimum from the assumption that the 2^n gaps between 1 and 2^n are equiprobable.
- In general, we do not know the **probability distribution over gaps a priori**.

Gamma codes ...

- The characteristic of a discrete probability distribution P that determines its coding properties (including whether a code is optimal) is its entropy $H(P)$.

$$H(P) = - \sum_{x \in X} P(x) \log_2 P(x)$$

- Entropy is a measure of uncertainty for a probability distribution P over two possible outcomes, namely, $X = \{x_1, x_2\}$.
- Entropy is maximized ($H(P)=1$) for $P(x_1) = P(x_2) = 0.5$ when uncertainty about which x_i will appear next is largest
- Entropy is minimized ($H(P)=0$) for $P(x_1)=1, P(x_2)=0$ and for $P(x_1)=0, P(x_2)=1$ when there is absolute certainty.



Gamma codes ...

- It can be shown that the lower bound for the expected length $E(L)$ of a code L is $H(P)$ if certain conditions hold (see the references).
- It can further be shown that for $1 < H(P) < \infty$, Gamma encoding is within a factor of 3 of this optimal encoding, approaching 2 for large $H(P)$:

$$\frac{E(L_\gamma)}{H(P)} \leq 2 + \frac{1}{H(P)} \leq 3.$$

- A code like Gamma code with the property of being within a factor of optimal for an arbitrary distribution P is called universal.

Gamma codes ...

- In addition to universality, Gamma codes have two other properties that are useful for index compression.
 - **Prefix free:** no gamma code is the prefix of another. This means that there is always a unique decoding of a sequence of gamma codes - and we do not need delimiters between them, which would decrease the efficiency of the code.
 - **Parameter free:** For many other efficient codes, we have to fit the parameters of a model (e.g., the binomial distribution) to the distribution of gaps in the index.
- **How much compression of inverted index do gamma codes achieve?**

$$c = \frac{1}{H_M} \approx \frac{1}{\ln M} = \frac{1}{\ln 400,000} \approx \frac{1}{13}$$

$$L_{\frac{c}{i}} \approx \frac{200 \times \frac{1}{13}}{i} \approx \frac{15}{i}$$

$$\begin{aligned} \text{bits-per-row} &= \frac{N}{j} \times (2 \times \lfloor \log_2 j \rfloor + 1) \\ &\approx \frac{2N \log_2 j}{j} \end{aligned}$$

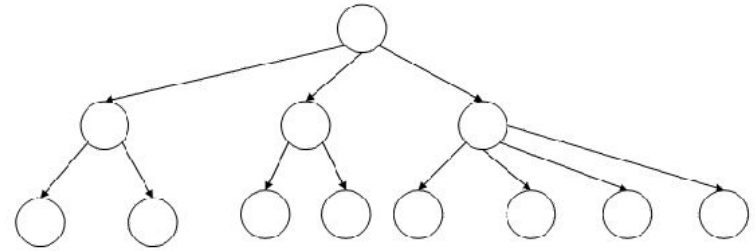
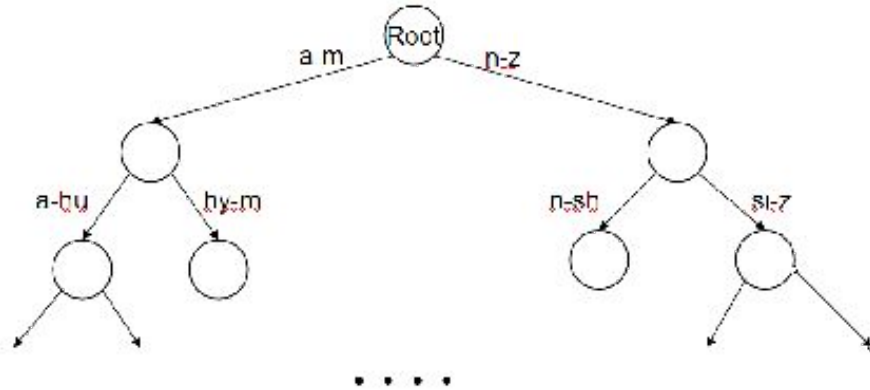
$$\sum_{j=1}^{27,000} \frac{2 \times 10^6 \times 15 \log_2 j}{j} \approx 224 \text{ MB.}$$

	N documents
Lc most frequent terms	N gaps of 1 each
Lc next most frequent terms	N/2 gaps of 2 each
Lc next most frequent terms	N/3 gaps of 3 each
...	...

- This **vocabulary lookup operation** uses a classical data structure called the dictionary.
- It has two broad classes of solutions: **hashing, and search trees**.
- In the literature of data structures, the entries in the vocabulary (in our case, terms) are often referred to as keys.
- The choice of solution (hashing, or search trees) is governed by a number of questions:
 - How many keys are likely to have?
 - Is the number likely to remain static or change a lot or to also have some keys in the dictionary be deleted?
 - What are the relative frequencies with which various keys will be accessed?

Search structures for dictionaries ...

- **Hashing** has been used for dictionary lookup in some search engines.
- **A binary search tree:** In this example the branch at the root partitions vocabulary terms into two subtrees, those whose first letter is between a and m, and the rest.
- **A B-tree:** In this example every internal node has between 2 and 4 children.

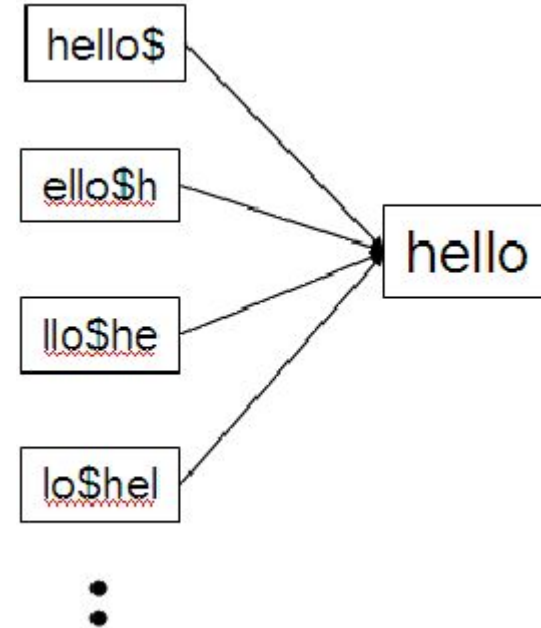


- We developed the ideas underlying inverted indexes for handling **Boolean and proximity** queries.
- We develop techniques that are robust to **typographical errors** in the query, as well as **alternative spellings**.
- We develop data structures that help the search for terms in the vocabulary in an inverted index.
- **Wildcard query:** a query such as **a*e*i*o*u**, which seeks documents containing any term that includes all the five vowels in sequence.
 - The *** symbol indicates any (possibly empty) string of characters.
- Users pose such queries to a search engine when they are uncertain about how to spell a query term, or seek documents containing variants of a query term.
- For example: the query **automat*** would seek documents containing any of the terms **automatic**, **automation** and **automated**.

- Wildcard queries are used in any of the following situations:
 - the user is uncertain of the **spelling of a query term**
 - the user is aware of **multiple variants of spelling a term** and (consciously) seeks documents containing any of the variants
 - the user seeks documents containing variants of a term that would be **caught by stemming**, but is unsure whether the search engine performs stemming
 - the user is uncertain of the **correct rendition** of a foreign word or phrase
- Types of wildcard queries: trailing, leading, general wildcard
- We can handle more general cases using a regular B-tree together with a reverse B-tree
- General wildcard queries
 - Express the given wildcard query q_w as a Boolean query Q on a specially constructed index, such that the answer to Q is a superset of the set of vocabulary terms matching q_w .
 - We check each term in answer to Q against q_w , discarding those vocabulary terms that do not match q_w .
 - At this point we have the vocabulary terms matching q_w and can resort to the standard inverted index.

Permuterm indexes

- Our first special index for general wildcard queries is **permuterm index** into our character set, to mark the end of a term.
- The term “**hello**” is shown here as the augmented term hello\$.
- Next, we construct a permuterm index, in which the various rotations of each term (augmented with \$) all link to the original vocabulary term.
- We refer to the set of rotated terms in the permuterm index as the **permuterm vocabulary**.
- How does this index help us with wildcard queries?
 - Consider the wildcard query m*n.
 - The key is to rotate such a wildcard query so that the * symbol appears at the end of the string - thus the rotated wildcard query becomes n\$m*.
 - Next, we look up this string in the permuterm index, where seeking n\$m* (via a search tree) leads to rotations of (among others) the terms man and moron.



- Now that the permuterm index enables us to **identify the original vocabulary terms** matching a wildcard query, we look up these terms in the standard inverted index to retrieve matching documents.
- We can thus handle any wildcard query with a single * symbol.
- But what about a query such as **fi*mo*er**?
- In this case we **first enumerate the terms** in the dictionary that are in the permuterm index of er\$fi*.

K-gram indexes for wildcard queries

- **K-gram** is a sequence of k characters.
- We can use a special character \$ to denote beginning and ending of term.
- Full set of 3-grams for “castle” is :\$ca, cas, ast, stl, tle, le\$
- In k-gram index, dictionary contains all k-grams that occur in any term in vocabulary.
- Each postings list points from a k-gram to all vocabulary terms containing that k-gram.
- For example, the 3-gram “etr” would point to vocabulary terms such as “metric” and “retrieval”



K-gram indexes for wildcard queries ...

Consider using 3-gram index described for query red*:

- First issue boolean query \$re AND red to 3-gram index may result “retired”
- **Post-filtering step:** in which terms enumerated by boolean query on 3-gram index are checked individually against original query red* through string matching.
- Search engines allow combination of wild card queries using boolean operators for example “red*d AND fe*ri”

- We next look at the problem of correcting spelling errors in queries.
- Two steps to solving this problem: Edit distance and k-gram overlap.
- There are two basic principles underlying most spelling correction algorithms.
 - **For a misspelled query, choose the “nearest” one.**
 - This demands that we have a notion of nearness or proximity between a pair of queries.
 - **When two correctly spelled queries are tied (or nearly tied), select the one that is more common.**
 - For instance, “grunt” and “grant” both seem equally plausible as corrections for “grnt”.

Spelling correction

- Types of spelling errors:
 - Non-word errors
 - graffe □ giraffe
 - Real-word errors
 - Typographical errors
 - three □ there
 - Cognitive Errors (homophones)
 - Piece □ peace
 - too □ two
 - your □ you're
- Forms of spelling correction
 - Non-word correction was historically mainly context insensitive
 - Isolated-term correction : Edit distance, k-gram
 - Real-word correction almost needs to be context sensitive
 - Context-sensitive correction

Edit distance:

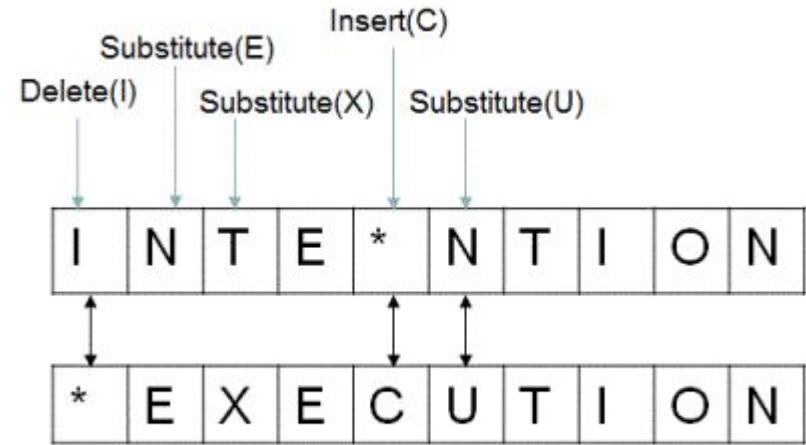
- Given two character strings s_1 and s_2 , the edit distance between them is the minimum number of edit operations required to transform s_1 into s_2 .
- Most commonly, the edit operations allowed for this purpose are:
 - (i) insert a character into a string;
 - (ii) delete a character from a string and
 - (iii) replace a character of a string by another character.
- Edit distance is sometimes known as **Levenshtein distance**.
- For example, the edit distance between “cat” and “dog” is 3.

Edit distance example

Dynamic programming algorithm for computing the edit distance between strings s_1 and s_2 .

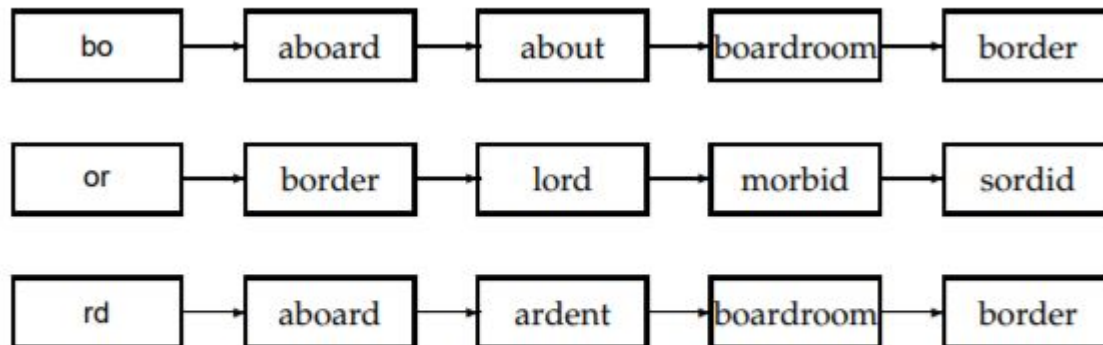
EDITDISTANCE(s_1, s_2)

```
1  int  $m[i, j] = 0$ 
2  for  $i \leftarrow 1$  to  $|s_1|$ 
3  do  $m[i, 0] = i$ 
4  for  $j \leftarrow 1$  to  $|s_2|$ 
5  do  $m[0, j] = j$ 
6  for  $i \leftarrow 1$  to  $|s_1|$ 
7  do for  $j \leftarrow 1$  to  $|s_2|$ 
8      do  $m[i, j] = \min\{m[i-1, j-1] + \text{if } (s_1[i] = s_2[j]) \text{ then } 0 \text{ else } 1, \text{if}$ 
9           $m[i-1, j] + 1,$ 
10          $m[i, j-1] + 1\}$ 
11  return  $m[|s_1|, |s_2|]$ 
```



k-gram indexes for spelling correction

- K-gram index assist with retrieving vocabulary terms with low edit distance to the query q .
- We can then find the ones of least edit distance from q .
- Ex: matching at least two of the three 2-grams in the query “**bord**”.



- Isolated-term correction would fail to correct **typographical errors**
 - Ex: “flew form Heathrow”, where all three query terms are correctly spelled.
- The simplest way is
 - to enumerate corrections of each of the three query terms even though each query term is correctly spelled
 - “fled form Heathrow” and “flew fore Heathrow”
 - then try substitutions of each correction in the phrase.
 - For each such substitute phrase, the search engine runs the query and determines the number of matching results.
- **Several heuristics** are used to trim this space.
 - We retain only the most frequent combinations in the collection or in the query logs, which contain previous queries by users.

Thank you